

Trabajo de Fin de Máster presentado en la Escuela Técnica Superior de Ingeniería de la Universidad de Santiago de Compostela, en cumplimiento de los requisitos para la obtención del título de Máster Universitario en Tecnologías de Análisis de Datos Masivos: Big Data.

Dr. Matías FERNÁNDEZ LAKATOS

Aplicación de nuevos modelos de detección de anomalías en contextos Big Data

DIRECTORES DEL TRABAJO

Dr. Victor J. GALLEGO FONTENLA Universidad de Santiago de Compostela

Msc. Pablo CEREIJO GARCÍA Centro Tecnológico De Telecomunicaciones De Galicia

SANTIAGO DE COMPOSTELA, GALICIA, ESPAÑA

1 de julio de 2025

"La realidad no es determinista; es una muestra. Cada observación es una mirada fugaz a una distribución de probabilidad de alta dimensión, que los autoencoders variacionales intentan aproximar, no memorizar. En esta visión, los outliers no son errores, sino realizaciones de baja probabilidad de la misma verdad generativa."

- Matías Fernández Lakatos

AGRADECIMIENTOS

Contemplar una pintura desde la distancia, especialmente una de estilo impresionista, nos permite apreciar una imagen completa y armoniosa. De forma análoga, esta memoria representa una visión integral de mi trabajo. Pero si nos acercamos, distinguimos innumerables pinceladas aparentemente inconexas: cada una de ellas simboliza los gestos de apoyo, las conversaciones, los consejos y los vínculos que han dado forma a este camino. Con esta metáfora en mente, quiero expresar mi más profundo agradecimiento a las personas e instituciones que han sido parte esencial de esta obra.

A mi familia más cercana: mis padres, Marión Victoria Lakatos y Álvaro Fernández; mi hermana, Maite; y mi tía Marta. Su amor incondicional, su guía y su confianza han sido pilares fundamentales en mi desarrollo personal y académico.

A mi esposa, Cecilia Pedemonte, compañera de vida y apoyo constante en este camino. Su paciencia, comprensión y fortaleza han sido un pilar fundamental para completar este trabajo. Este trabajo también es suyo.

A mi hija, Fran, que me inspira cada día a seguir adelante y a ser el ejemplo de curiosidad, perseverancia y libertad que deseo para ella.

A mis tutores, Pablo Cereijo y Víctor Gallego, por orientarme con generosidad y paciencia en este nuevo sendero hacia el mundo de la ciberseguridad. A los profesores de la Universidad de Santiago, quienes me brindaron las herramientas conceptuales y técnicas para enfrentarme con confianza a desafíos más complejos.

A los distinguidos miembros del tribunal, por asumir la responsabilidad de evaluar este trabajo y enriquecerlo con sus observaciones.

Agradezco también el apoyo económico proporcionado por la Xunta de Galicia, y muy especialmente a la Secretaría Xeral da Emigración, por su Programa de Becas Excelencia Juventud Exterior (BEME), que hizo posible mi formación en esta etapa.

Y a todas las personas que, de una forma u otra, estuvieron presentes en mi día a día, ofreciendo palabras de ánimo, escucha o compañía silenciosa: gracias.

A todas estas personas e instituciones, mi más sincero agradecimiento. Sin su ayuda, este trabajo no habría sido posible.

A Francisca y Cecilia

ABSTRACT

This work addresses the problem of early anomaly detection in corporate cybersecurity environments, where the volume and velocity of data hinder the application of traditional solutions. The project is set within a realistic context of network traffic and log analysis that monitors user behavior with respect to applications, enabling the identification of suspicious patterns in UEBA (User and Entity Behavior Analytics) scenarios. In this context, a β -Variational Autoencoder (β -VAE) model is proposed and developed for anomaly detection, featuring a mean-field variational inference approach and capable of modeling complex behaviors through a probabilistic latent space. The solution is implemented on a scalable architecture based on Big Data ecosystem technologies such as Kafka, Spark, and Airflow.

Unlike traditional autoencoders, the VAE provides advantages in latent space regularization and in quantifying the uncertainty associated with each reconstruction, which improves the interpretability and explanation of detected anomalies.

The model was implemented in a modular and flexible manner, facilitating tuning and scalability, and was evaluated using public cybersecurity datasets. The results show competitive performance in anomaly detection, surpassing classical approaches in certain aspects and clearly demonstrating its potential for integration into streaming data processing pipelines.

In addition, the model was integrated into a professional and reproducible infrastructure capable of operating in real time over large-scale data streams, representing a significant contribution to automated threat detection in Big Data environments. Finally, future work is outlined, focusing on real-world data application, performance optimization, and improved accuracy through advanced probabilistic interpretability techniques.

Keywords: Anomaly detection, Variational Autoencoder, UEBA, Cybersecurity, Big Data, Unsupervised learning, Neural networks, Behavior analysis, Interpretability, Streaming pipeline.

RESUMEN

En la presente memoria se aborda el problema de la detección temprana de anomalías en entornos de ciberseguridad corporativa, donde el volumen y la velocidad de los datos dificultan la aplicación de soluciones tradicionales. El trabajo se enmarca en un contexto realista de análisis de tráfico de red y logs que monitorizan el comportamiento de los usuarios frente a aplicaciones, lo cual permite identificar patrones sospechosos en escenarios UEBA (User and Entity Behavior Analytics). En este contexto, se propone y desarrolla un modelo de Autoencoder Variacional β (β -VAE) con una aproximación de inferencia tipo *mean-field*, orientado a la detección de anomalías y capaz de modelar comportamientos complejos mediante un espacio latente probabilístico. La solución se implementa sobre una arquitectura escalable basada en tecnologías del ecosistema Big Data como Kafka, Spark y Airflow.

A diferencia de los autoencoders clásicos, el VAE ofrece ventajas en la regularización del espacio latente y en la cuantificación de la incertidumbre asociada a cada reconstrucción, lo cual mejora la interpretabilidad y explicación de las anomalías detectadas.

El modelo fue implementado de forma modular y flexible, facilitando su ajuste y escalabilidad, y se evaluó usando conjuntos de datos públicos representativos de ciberseguridad. Los resultados demuestran un desempeño competitivo en la detección de anomalías, superando en ciertos aspectos a enfoques clásicos y mostrando un potencial claro para su aplicación en pipelines de procesamiento en streaming.

Además, se integró el modelo dentro de una infraestructura profesional y reproducible, capaz de operar en tiempo real sobre flujos masivos de datos, lo que representa un aporte significativo para la detección automática de amenazas en entornos Big Data. Finalmente, se plantean futuras líneas de trabajo orientadas a la aplicación en datos reales, optimización del rendimiento y mejora de la precisión mediante técnicas avanzadas de interpretación probabilística.

Palabras claves: Detección de anomalías, Autoencoder Variacional, UEBA, Ciberseguridad, Big Data, Aprendizaje no supervisado, Redes neuronales, Análisis de comportamiento, Interpretabilidad, Pipeline en streaming.

ÍNDICE GENERAL

ABSTRACT	ix
RESUMEN	xi
1 INTRODUCCIÓN	1
2 REQUISITOS DE LA APLICACIÓN	5
2.1 Requisitos Funcionales	5
2.2 Requisitos No Funcionales	6
2.3 Infraestructura de Almacenamiento y Persistencia	6
3 ANÁLISIS DE SOLUCIONES EXISTENTES Y REVISIÓN DEL ESTADO DEL ARTE	7
3.1 Detección de anomalías en entornos UEBA y Big Data	7
3.2 Métodos clásicos de detección de anomalías	7
3.3 Autoencoders y su evolución en entornos de alta complejidad	8
3.4 Autoencoders Variacionales: modelos generativos para Big Data	9
3.5 Comparativa y visión integradora	10
4 PROPUESTA DE SOLUCIÓN Y GRADO DE INNOVACIÓN	13
4.1 Arquitectura del Modelo	13
4.2 Espacio Latente	16
4.3 Función de pérdida	17
4.3.1 Interpretación del factor β	17
4.4 Técnicas de escalado de características	18
4.5 Entrenamiento del Modelo	19
4.6 Evaluación del Modelo	20
4.6.1 Interpretación de las Métricas de Evaluación y del Umbral	21
5 RESULTADOS Y ANÁLISIS	23
5.1 Descripción del conjunto de datos	23
5.2 Preprocesamiento de los datos	23
5.3 Entrenamiento del modelo	23
5.4 Selección del umbral de reconstrucción	24
5.4.1 Evaluación sobre el conjunto de prueba	25
5.5 Búsqueda de Hiperparámetros: Grid Search y Random Search	26
5.6 Comparación con otros modelos	28
6 CONCLUSIONES Y TRABAJO A FUTURO	31
6.1 Conclusiones	31
6.2 Trabajo a Futuro	31
A FUNDAMENTO TEÓRICO	33
A.1 Fundamentos probabilísticos del Autoencoder Variacional	33
A.2 Modelo latente generativo	33
A.3 Inferencia Variacional Amortiguada	34
A.4 Optimización de la ELBO y truco de la reparametrización	35
A.5 Amortización y suposiciones sobre la posterior	36
A.6 Correspondencia entre la función de pérdida y la ELBO	37

B	INICIALIZACIÓN DE PESOS	39
B.1	Resumen de esquemas utilizados	39
B.2	He Normal (para ReLU)	39
B.3	Glorot Uniform (para z_{mean})	40
B.4	LeCun Normal (para $z_{\text{log_var}}$)	40
C	EXPLORACIÓN DETALLADA DEL CONJUNTO DE DATOS	41
C.1	Análisis exploratorio de características	41
C.1.1	Distribuciones diferenciadas por clase	41
C.1.2	Relación multivariada con la variable objetivo	44
D	DETECCIÓN DE ANOMALÍAS Y RECONSTRUCCIÓN	45

INTRODUCCIÓN

En la era del Big Data, la detección temprana de anomalías se ha convertido en una necesidad crítica para garantizar la seguridad y la estabilidad de los sistemas digitales. En particular, en entornos de ciberseguridad corporativa, el volumen, variedad y velocidad de los datos dificulta la implementación de mecanismos de defensa tradicionales. Esta memoria se enmarca en un escenario realista de análisis de tráfico de red y registros de eventos (logs) en tiempo real, incluyendo logs de sistemas, aplicaciones y dispositivos de seguridad, los cuales aportan información clave para la detección de comportamientos anómalos. Para ello, se procesan grandes flujos de datos utilizando una arquitectura escalable basada en tecnologías modernas del ecosistema Big Data. Este trabajo forma parte del proyecto titulado *Centro de operaciones de seguridad basado en UEBA explicable* (CPP001/23_R06_SafeNet_UEBA), con el acrónimo *SafeNet UEBA*, financiado por la Unión Europea a través de los fondos *NextGenerationEU*, en el marco del *Plan de Recuperación, Transformación y Resiliencia*, gestionado por INCIBE[†].

Los datos brutos —obtenidos a través de APIs como Microsoft Graph Security o mediante captura directa de tráfico de red— son extraídos en streaming, publicados en un clúster de Kafka y almacenados en MinIO[‡] para persistencia distribuida. A partir de allí, son procesados mediante Apache Spark para llevar a cabo tareas de limpieza, transformación y agregación, siendo posteriormente reenviados a Kafka para su consumo por distintos servicios analíticos. Python se utiliza como lenguaje principal para el entrenamiento de modelos de aprendizaje automático, entre ellos el modelo central de este trabajo. Todo este pipeline de procesamiento está orquestado mediante Apache Airflow[§], permitiendo una ejecución modular, reproducible y escalable de cada etapa del flujo de trabajo.

En este contexto, la detección de anomalías no puede abordarse con soluciones estáticas o rígidas. Es necesario aplicar enfoques adaptativos y capaces de modelar comportamientos complejos, como lo permite la *Analítica del Comportamiento de Usuarios y Entidades* (UEBA, por sus siglas en inglés: *User and Entity Behavior Analytics*). UEBA combina técnicas de *inteligencia artificial*, análisis estadístico y reglas basadas en comportamiento para identificar desviaciones respecto a un patrón considerado “normal”. A diferencia de los sistemas tradicionales basados en firmas o reglas fijas, UEBA se centra en entender el comportamiento típico de usuarios y entidades para detectar actividades anómalas que podrían indicar amenazas internas, accesos indebidos o movimientos laterales en la red.

Entre los enfoques no supervisados más utilizados en la detección de anomalías se encuentran los *autoencoders* (AE) [1], redes neuronales que aprenden una representación comprimida de los datos mediante dos componentes: un *encoder*, que proyecta las entradas a un espacio latente de menor dimensión, y un *decoder*, que reconstruye la entrada original a partir de dicha representación. Al entrenarse minimizando el error de reconstrucción —generalmente mediante el error cuadrático medio (MSE)—, estos modelos capturan patrones frecuentes en los

[†] Información disponible sobre este proyecto en la página de INCIBE: <https://www.incibe.es/industria-cpi/ute-reto-6-ueba>

[‡] <https://min.io/>

[§] <https://airflow.apache.org/>

datos, lo que permite interpretar las observaciones con alto error como potenciales anomalías. Esta capacidad los convierte en herramientas útiles para tareas como la detección de intrusiones, fallos o comportamientos atípicos.

En este trabajo se explora específicamente el uso de un **Autoencoder Variacional β -VAE** [2], una extensión del VAE clásico [3, 4] que introduce un término de regularización escalado sobre la divergencia KL, controlado por un hiperparámetro $\beta > 0$. Este ajuste permite modular el compromiso entre fidelidad de reconstrucción y regularización del espacio latente: valores de $\beta > 1$ promueven una mayor compresión y disentanglement, mientras que valores $\beta < 1$ priorizan la reconstrucción. A diferencia del AE clásico, el VAE fuerza la codificación de los datos a seguir una distribución conocida (típicamente una normal multivariada). En particular, se adopta una aproximación de inferencia tipo *mean-field*, donde la distribución posterior aproximada se modela como una Gaussiana diagonal, lo que permite una optimización computacionalmente viable mediante el enfoque de variational Bayes. Esta estructura probabilística contribuye a un espacio latente continuo y regularizado, donde las anomalías pueden detectarse tanto por errores de reconstrucción como por desviaciones respecto a regiones de alta probabilidad latente.

Además, al tratarse de un modelo generativo probabilístico, el VAE permite cuantificar la incertidumbre asociada a cada reconstrucción. Esta incertidumbre puede evaluarse tanto a nivel de instancia como de atributo individual (feature-wise), brindando una herramienta valiosa para explicar no solo que un evento es anómalo, sino también en qué variables específicas el modelo presenta mayor dispersión o inconsistencia en la reconstrucción. Esta capacidad mejora significativamente la interpretabilidad del modelo en escenarios sensibles como la ciberseguridad, donde comprender el origen de una anomalía es tan importante como detectarla.

Cabe destacar que, aunque los VAE ofrecen ventajas teóricas al modelar explícitamente la distribución subyacente de los datos, su rendimiento no siempre supera al de los AE clásicos. En ciertos conjuntos de datos, especialmente aquellos con estructuras simples o donde el error de reconstrucción por sí solo es un indicador suficiente y directo de anomalía, los AE pueden mostrar un comportamiento más preciso al no estar condicionados por la regularización sobre el espacio latente.

No obstante, el uso de modelos generativos como el VAE aporta una perspectiva complementaria al problema: permite generar nuevas muestras realistas, explorar la distribución aprendida y detectar outliers como puntos de baja probabilidad en ese espacio. Estas propiedades resultan especialmente valiosas en escenarios como UEBA, donde los patrones normales de comportamiento pueden ser altamente complejos, cambiantes y de difícil modelado con enfoques puramente discriminativos o basados en reglas.

En este estudio, se propone una arquitectura escalable y modular para el entrenamiento y evaluación de un VAE aplicado a la detección de anomalías, con énfasis en la interpretación del error de reconstrucción y su utilidad en contextos reales de seguridad. Asimismo, se analiza comparativamente su desempeño frente al AE clásico, evaluando las condiciones bajo las cuales cada enfoque puede resultar más adecuado.

Objetivo general: Desarrollar una arquitectura escalable y reproducible para la detección de anomalías basada en autoencoders variacionales, aplicable a entornos UEBA, y validarla inicialmente sobre un conjunto de datos públicos de ciberseguridad, con el fin de facilitar su futura implementación sobre flujos de datos reales mediante tecnologías del ecosistema Big Data.

Objetivos específicos:

- Investigar y desarrollar un modelo de Autoencoder Variacional para la detección de anomalías en grandes volúmenes de datos generados por usuarios y entidades dentro de una organización.
- Diseñar e implementar un modelo VAE modular, con capacidad de ajuste flexible de su arquitectura y sus hiperparámetros, adaptable a datos no supervisados en escenarios de ciberseguridad.

- Analizar el comportamiento del error de reconstrucción y su relación con patrones anómalos, mejorando la interpretación del modelo y su valor explicativo.
- Evaluar el rendimiento del modelo utilizando un conjunto de datos públicos representativos de entornos UEBA, con tráfico y eventos realistas.
- Aplicar técnicas de visualización de datos que permitan representar de manera comprensible las predicciones del modelo y las amenazas detectadas, facilitando el análisis por parte de los equipos de ciberseguridad.
- Integrar el modelo en una base de código más amplia, organizada de manera modular, que permita su reutilización, escalabilidad y fácil mantenimiento dentro de pipelines de procesamiento más complejos.
- Adaptar el modelo a un pipeline de procesamiento distribuido y orientado a tiempo real, evaluando su funcionamiento en condiciones similares a las de producción.
- Mejorar la explicabilidad del modelo mediante el desarrollo de mecanismos que permitan comprender el porqué de sus predicciones, favoreciendo una toma de decisiones informada.
- Optimizar el rendimiento del sistema completo en entornos de Big Data, asegurando su capacidad de procesar flujos masivos de datos en tiempo real con bajo impacto en la latencia y los recursos.

Estructura del documento: El [Capítulo 1](#) introduce el problema que se aborda y contextualiza su relevancia. A continuación, el [Capítulo 2](#) describe los requisitos funcionales, no funcionales y de infraestructura que debe cumplir la aplicación para su despliegue en entornos de alta demanda como los Centros de Operaciones de Seguridad (SOC), incluyendo una arquitectura basada en capas de datos *bronze*, *silver* y *gold*, gestión de modelos con MLflow, persistencia en MinIO y observabilidad del sistema. Luego, el [Capítulo 3](#) presenta una revisión del estado del arte, incluyendo soluciones comerciales y académicas actualmente existentes. El [Capítulo 4](#) expone la propuesta metodológica basada en autoencoders variacionales, destacando el grado de innovación que representa junto al desarrollo técnico y la descripción de los aspectos clave de implementación de dicha propuesta. Posteriormente, el [Capítulo 5](#) presenta los resultados obtenidos y su análisis en profundidad, finalizando con una comparación entre el modelo Autoencoder clásico, Isolation Forest y Extended Isolation Forest. Finalmente, el [Capítulo 6](#) resume las conclusiones del trabajo y plantea posibles líneas de investigación futura.

El [Apéndice A](#) complementa el documento incluyendo el fundamento teórico que sustenta la solución propuesta, mientras que el [Apéndice B](#) profundiza en la elección de la inicialización de los pesos del modelo en el entrenamiento. Por su parte, el [Apéndice C](#), desarrolla un análisis detallado de la exploración y caracterización del conjunto de datos utilizado. Finalmente, el [Apéndice D](#) presenta un análisis comparativo del comportamiento del modelo con diferentes configuraciones del hiperparámetro β , poniendo especial énfasis en la relación entre capacidad de reconstrucción y eficacia en la detección de anomalías.

En total, esta memoria cuenta con 30 páginas de desarrollo principal y 16 páginas adicionales de apéndices complementarios que profundizan aspectos técnicos y experimentales relevantes. Además, se incluyen un resumen en español y un abstract en inglés para facilitar su difusión en contextos académicos nacionales e internacionales.

REQUISITOS DE LA APLICACIÓN

Este trabajo se enmarca dentro de una arquitectura más amplia orientada a la detección de anomalías en entornos de alta exigencia como los SOC. Si bien los requisitos aquí expuestos describen el conjunto de capacidades que debe tener la aplicación en su totalidad, el desarrollo presentado en esta memoria se centra particularmente en la construcción, entrenamiento y despliegue de un modelo de β -VAE. Para que dicho modelo pueda cumplir su función de detección automática de eventos anómalos, es imprescindible que la arquitectura asegure la disponibilidad de datos en tiempo real, en el formato adecuado, sin valores faltantes, así como una infraestructura de almacenamiento que permita tanto el entrenamiento eficiente sobre grandes volúmenes de datos como la posterior persistencia de los modelos y sus predicciones. Esta integración entre componentes puede observarse en la [Figura 2.1](#), donde se ilustra el flujo completo desde la ingesta de datos hasta la generación y almacenamiento de alertas. En este sentido, los requisitos descritos a continuación no solo representan condiciones generales del sistema, sino también prerequisites clave para la efectiva implementación de la solución propuesta en este trabajo.

2.1 REQUISITOS FUNCIONALES

- **Captura de datos en tiempo real:** la solución debe recibir datos de actividad del sistema en tiempo real desde diversas fuentes (logs, eventos, etc.).
- **Procesamiento continuo:** el sistema debe integrarse en un pipeline ETL para procesar, transformar y enriquecer los datos de manera continua mediante Kafka.
- **Detección de anomalías:** la aplicación debe realizar inferencia en línea usando un modelo entrenado, y marcar eventos sospechosos o anómalos.
- **Entrenamiento periódico o continuo:** debe poder reentrenar modelos con nuevos datos, según necesidades operativas o degradación del rendimiento.
- **Gestión de modelos:** se requiere una integración con herramientas como MLflow para la gestión de versiones de los modelos, sus métricas y sus artefactos.
- **Almacenamiento de resultados:** las predicciones y alertas deben ser persistidas para su posterior análisis en una base de datos del SOC.
- **Orquestación de flujos de trabajo:** la solución debe utilizar una herramienta de orquestación, como Apache Airflow, para coordinar y automatizar las tareas de ingesta, procesamiento, entrenamiento, inferencia y almacenamiento de datos.

2.2 REQUISITOS NO FUNCIONALES

- **Integración con infraestructura existente:** la solución debe integrarse con el stack tecnológico ya desplegado, incluyendo Kafka, MinIO y servicios en la nube.
- **Escalabilidad:** el sistema debe ser capaz de escalar horizontalmente para procesar grandes volúmenes de datos en contextos Big Data.
- **Tolerancia a fallos y recuperación:** debe contar con mecanismos de respaldo y recuperación para prevenir pérdida de datos o interrupciones críticas.
- **Interpretabilidad y trazabilidad:** las decisiones del modelo deben ser trazables y explicables, incluyendo la correlación con eventos previos o simultáneos.
- **Versatilidad en el almacenamiento:** aunque actualmente los datos se almacenan en OneDrive (entorno nube), el sistema debe poder adaptarse a otros entornos bajo solicitud del cliente.

2.3 INFRAESTRUCTURA DE ALMACENAMIENTO Y PERSISTENCIA

La arquitectura de datos se estructura en tres capas lógicas, siguiendo un enfoque moderno de gobernanza:

- **Capa Bronze:** contiene datos crudos tal como se reciben del sistema de origen. Estos datos son almacenados en MinIO, con respaldo periódico.
- **Capa Silver:** contiene datos transformados o agregados, resultado de procesos de limpieza y enriquecimiento. Estos datos pueden ser completamente reconstruidos desde la capa Bronze.
- **Capa Gold:** contiene resultados derivados de los modelos analíticos, como etiquetas de anomalías, indicadores o estadísticas. Se puede reconstruir desde la capa Silver.

El almacenamiento principal se realiza en MinIO, que actúa como repositorio central de datos y modelos. La persistencia incluye tanto datasets como artefactos de modelos entrenados, configuraciones, métricas y registros de entrenamiento. Además, se contempla el respaldo automático de la capa Bronze, para garantizar la recuperación ante fallos.

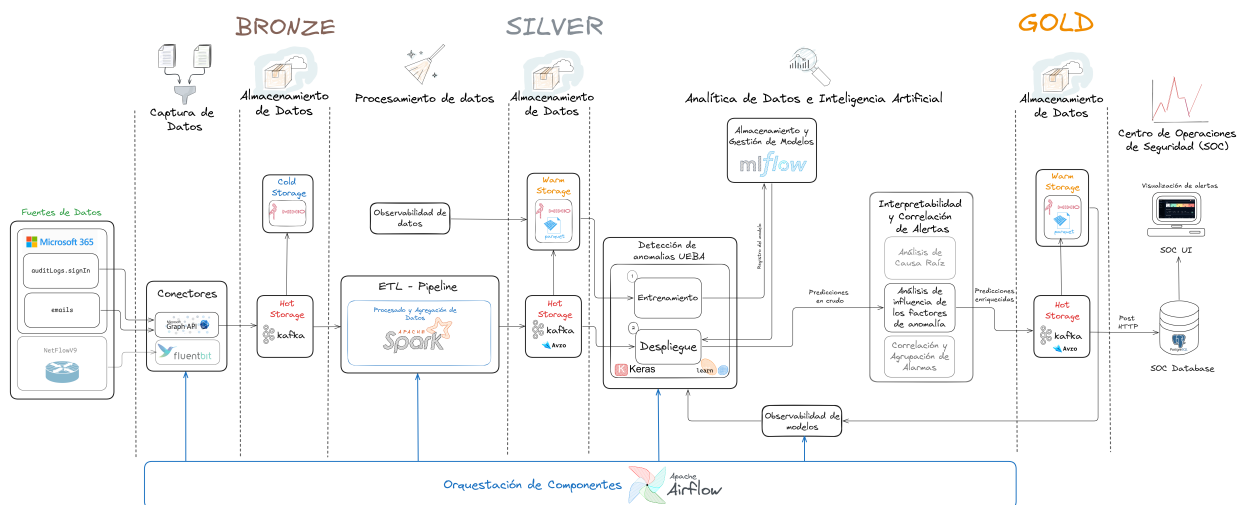


Figura 2.1: Arquitectura general del flujo de datos, entrenamiento, inferencia y almacenamiento.

ANÁLISIS DE SOLUCIONES EXISTENTES Y REVISIÓN DEL ESTADO DEL ARTE

Este capítulo presenta un análisis de las soluciones actualmente disponibles para la detección de anomalías en entornos de ciberseguridad, con énfasis en enfoques académicos consolidados y su aplicabilidad a escenarios reales, incluyendo sistemas UEBA y contextos Big Data. A continuación, se revisan las principales técnicas utilizadas en la literatura, desde métodos estadísticos clásicos hasta enfoques recientes basados en redes neuronales profundas, con especial atención a los Autoencoders y sus variantes. Esta revisión permite establecer una base comparativa sólida que justifica e interpreta la solución propuesta en los capítulos siguientes.

3.1 DETECCIÓN DE ANOMALÍAS EN ENTORNOS UEBA Y BIG DATA

El crecimiento acelerado de la digitalización, la computación en la nube y el IoT ha llevado a una explosión en la cantidad y variedad de datos generados por usuarios, sistemas y dispositivos. En este contexto, los sistemas de **UEBA** han ganado protagonismo como mecanismos clave para la detección de amenazas en ciberseguridad, particularmente en entornos de Big Data.

UEBA se basa en la caracterización del comportamiento normal de usuarios y entidades mediante análisis estadístico y aprendizaje automático. Su fortaleza radica en la capacidad de identificar desviaciones que podrían representar actividades maliciosas o negligentes [5, 6, 7]. A diferencia de los enfoques basados en firmas, UEBA es especialmente eficaz frente a ataques desconocidos o sutiles.

La naturaleza distribuida y heterogénea de los datos en entornos Big Data —registros de eventos, tráfico de red, datos de aplicaciones, sensores o dispositivos móviles— requiere modelos capaces de escalar y adaptarse a información en tiempo real, lo que ha impulsado la investigación en técnicas avanzadas de detección de anomalías.

3.2 MÉTODOS CLÁSICOS DE DETECCIÓN DE ANOMALÍAS

Los enfoques tradicionales para la detección de anomalías han sido fundamentales en etapas iniciales del análisis de datos. A diferencia de los modelos basados en redes neuronales, estos métodos no utilizan algoritmos de retropropagación (backpropagation) para ajustar parámetros, sino que se basan en técnicas estadísticas, reglas heurísticas o particiones estructuradas del espacio de datos [8]. Algunos de los más relevantes incluyen:

- **Modelos de regresión penalizada (Lasso, Ridge):** aplicados a tareas de detección mediante el análisis de residuos o desviaciones en variables objetivo. Su inferencia es muy rápida, pero su capacidad para capturar

relaciones no lineales o complejas es limitada [9, 10, 11]. Además, pueden requerir un ajuste cuidadoso de los hiperparámetros para evitar sobreajuste o infraajuste.

- **Máquinas de Vectores de Soporte (SVM):** aprende una frontera óptima que delimita la región de normalidad. Su tiempo de inferencia depende del número de vectores de soporte, que puede crecer con el tamaño del dataset, afectando su escalabilidad [12, 13]. Además, su desempeño puede verse limitado en presencia de datos muy ruidosos o altamente dimensionales.
- **Local Outlier Factor (LOF):** calcula la densidad local de un punto con respecto a sus vecinos, detectando como anomalías aquellos que presentan densidad significativamente menor. Aunque su entrenamiento es simple, la inferencia requiere calcular distancias a múltiples vecinos, lo que puede volverse costoso computacionalmente en grandes volúmenes de datos o alta dimensionalidad [14, 15]. Esto limita su aplicabilidad en escenarios Big Data.
- **K-Nearest Neighbors (KNN):** identifica anomalías en función de la distancia a sus vecinos más cercanos. Si una observación está lejos de sus k vecinos, se considera una anomalía. Aunque no requiere entrenamiento, su tiempo de inferencia es alto, especialmente en grandes datasets, ya que implica calcular distancias a muchos puntos. Esto puede ser crítico en contextos en línea o en tiempo real [16, 17, 15].
- **Árboles de decisión y Random Forests:** pueden adaptarse para detectar anomalías mediante umbrales de probabilidad o profundidad de los nodos. Son rápidos en inferencia, fácilmente interpretables y escalables mediante paralelización [18, 19, 20]. Sin embargo, pueden presentar dificultades para modelar patrones muy complejos o dinámicos sin una configuración adecuada.
- **Isolation Forest:** técnica basada en árboles de partición aleatoria que aísla observaciones a través de divisiones binarias. Las anomalías suelen requerir menos divisiones para ser aisladas. Su entrenamiento e inferencia son eficientes, incluso en grandes conjuntos de datos, lo que lo hace adecuado para Big Data [21]. Sin embargo, puede presentar limitaciones para detectar anomalías que dependen de interacciones complejas entre variables.
- **Extended Isolation Forest (EIF):** una mejora del Isolation Forest que utiliza particiones extendidas y una función de separación más flexible, mejorando la detección en datos con alta dimensionalidad y estructuras complejas. Aunque suele ofrecer mejor calidad en la detección, su tiempo de entrenamiento y ajuste puede ser considerablemente mayor que el Isolation Forest clásico [22].

A pesar de sus ventajas, estos enfoques presentan limitaciones en entornos con alta dimensionalidad, datos no estructurados o flujos en tiempo real, donde la capacidad para modelar relaciones complejas es crucial. Diversos estudios han mostrado que, si bien los métodos clásicos ofrecen resultados competitivos en contextos controlados, los modelos basados en aprendizaje profundo tienden a superarlos en escenarios más dinámicos o con datos altamente no lineales [23].

3.3 AUTOENCODERS Y SU EVOLUCIÓN EN ENTORNOS DE ALTA COMPLEJIDAD

En el contexto del proyecto UEBA, se optó por un enfoque no supervisado de detección de anomalías, dada la escasez de datos etiquetados que indiquen claramente comportamientos anómalos. Esta elección metodológica implica que los modelos deben aprender a identificar desviaciones sin información previa sobre lo que constituye una anomalía, lo cual es común en entornos de ciberseguridad. En contraste, los enfoques supervisados requieren un conjunto de entrenamiento con etiquetas explícitas, lo cual no siempre es factible en escenarios reales.

A medida que los entornos de análisis se tornan más dinámicos, distribuidos y multivariantes —como se detalló en la [Sección 3.1](#) con respecto a sistemas UEBA y Big Data—, la necesidad de modelos capaces de capturar relaciones no lineales y de adaptarse a estructuras complejas de datos se vuelve crítica. En este contexto, los AE emergen como una evolución natural frente a los métodos clásicos descritos en la [Sección 3.2](#), superando sus limitaciones de escalabilidad, linealidad y sensibilidad al ruido.

Un AE es una red neuronal que aprende a reconstruir su entrada a partir de una codificación intermedia, lo que implica extraer las estructuras subyacentes del conjunto de datos. Esta capacidad lo convierte en un modelo eficaz para la detección de anomalías, ya que las observaciones atípicas suelen presentar errores de reconstrucción significativamente más altos que las muestras normales [24, 25, 26].

La ventaja clave de los AE frente a métodos como LOF, SVM o Isolation Forest es su habilidad para modelar relaciones altamente no lineales y distribuciones complejas sin requerir una supervisión directa. Esto los vuelve ideales para contextos UEBA, donde las anomalías pueden ser sutiles, dispersas y mal definidas [27, 28]. Además, su entrenamiento en grandes volúmenes de datos sin etiquetas —una necesidad común en entornos Big Data— los posiciona como herramientas especialmente útiles para monitoreo continuo y análisis en tiempo real.

Diversas variantes de AEs se han desarrollado para abordar particularidades de los datos:

- Los **autoencoders convolucionales (CAE)** son eficaces en tareas donde la estructura espacial es relevante, como imágenes o tráfico de red representado como mapas de características [29].
- Los **autoencoders LSTM** extienden la arquitectura clásica de AEs a series temporales, utilizando redes neuronales recurrentes con unidades de memoria a largo plazo (LSTM). Esto les permite capturar dependencias temporales complejas y patrones secuenciales prolongados en el comportamiento de usuarios o dispositivos, fundamentales en entornos UEBA donde los datos son flujos de eventos ordenados en el tiempo. Gracias a esta capacidad, los AEs LSTM pueden aprender la dinámica normal de actividad y detectar anomalías que se manifiestan como desviaciones inusuales en la secuencia temporal de acciones [30, 31].
- Los **denoising autoencoders** incorporan ruido de forma intencional durante el entrenamiento para aprender representaciones robustas, útiles en escenarios con sensores ruidosos o registros incompletos [32].

Sin embargo, los AE tradicionales operan bajo una codificación determinista y carecen de una interpretación probabilística formal del espacio latente. Esta limitación impide, por ejemplo, establecer umbrales basados en la probabilidad de generación de una muestra o modelar explícitamente la incertidumbre. En aplicaciones como las descritas en la [Sección 3.1](#), donde el comportamiento puede ser ambiguo, variable o parcialmente observado, esta carencia puede restringir la capacidad adaptativa del modelo [33].

Estas deficiencias motivaron el desarrollo de enfoques generativos probabilísticos como los VAE, tratados en detalle en la siguiente sección.

3.4 AUTOENCODERS VARIACIONALES: MODELOS GENERATIVOS PARA BIG DATA

Los **VAE** [4] constituyen una evolución de los AEs clásicos al incorporar una perspectiva bayesiana. En lugar de aprender una codificación determinista, los VAE modelan una distribución latente probabilística, lo que permite generar nuevas muestras y calcular la probabilidad de observaciones individuales. Esta capacidad es fundamental para establecer umbrales probabilísticos robustos en la detección de anomalías, especialmente en escenarios complejos y dinámicos.

En entornos UEBA, donde el comportamiento de usuarios y dispositivos es altamente variable y contextual, las anomalías pueden manifestarse como desviaciones sutiles, graduales o dependientes del contexto temporal y

de interacciones complejas. Esto dificulta su detección mediante métodos puramente deterministas o basados en umbrales rígidos, que suelen asumir patrones estáticos y claras rupturas en los datos.

Los **VAE** facilitan la adaptación continua al modelar un espacio latente probabilístico, continuo y estructurado que representa la variabilidad legítima del comportamiento. De este modo, permiten mejorar la detección de anomalías que no serían evidentes con métodos tradicionales, identificando patrones atípicos en regiones de baja probabilidad dentro del espacio latente [34].

Una variante particularmente relevante es el β -VAE [2], que introduce un hiperparámetro $\beta > 0$ para ajustar el equilibrio entre reconstrucción y regularización del espacio latente. Esta modificación permite controlar la capacidad del modelo para generalizar y descomponer factores de variación, lo que resulta útil en contextos donde es necesario un mayor control sobre la compresibilidad o interpretabilidad del espacio latente.

Adicionalmente, la arquitectura modular de los VAE y su compatibilidad con plataformas de cómputo distribuido (por ejemplo, TensorFlow Extended o PyTorch en GPU/TPU) los hacen adecuados para su aplicación en Big Data. Recientes investigaciones integran VAE con mecanismos de atención (transformers) o sistemas de procesamiento de streaming, habilitando detección en tiempo real y escalabilidad horizontal [35].

3.5 COMPARATIVA Y VISIÓN INTEGRADORA

En la Tabla 3.1 se sintetizan las características, ventajas y limitaciones de diferentes enfoques para detección de anomalías, enfatizando su aplicabilidad en contextos de Big Data y UEBA.

- **Modelos clásicos** (PCA, LOF, Isolation Forest, etc.) se destacan por su rapidez y bajo costo computacional, siendo ideales como líneas base o para preprocesamiento. Sin embargo, presentan limitaciones para captar relaciones no lineales complejas o adaptarse a escenarios altamente dinámicos.
- **AE** ofrecen una mayor capacidad para aprender representaciones no lineales y complejas, siendo útiles en contextos semi-supervisados y para datos de alta dimensión. No obstante, requieren cuidado en el diseño y entrenamiento para evitar sobreajuste.
- **AEs LSTM** amplían esta capacidad al modelar secuencias temporales, capturando dependencias a largo plazo en el comportamiento de usuarios o dispositivos, lo que resulta esencial en UEBA donde la información temporal y contextual es clave para detectar anomalías.
- **VAE** y β -VAE agregan un enfoque generativo y probabilístico, permitiendo no solo detección más fina y adaptable, sino también generación de escenarios y simulaciones que mejoran la robustez del sistema. Son particularmente efectivos en datos secuenciales y flujos continuos propios de Big Data.

Es fundamental destacar que, dada la complejidad y volumen de datos en entornos Big Data, la integración de técnicas generativas avanzadas como los VAE con modelos más interpretables y eficientes en etapas preliminares, conforma una estrategia híbrida que maximiza tanto la precisión como la escalabilidad.

Modelo	Tipo	Ventajas Principales	Limitaciones	Ejemplo de Uso
Mahalanobis + SVD	Estadístico	Fácil de interpretar; bajo costo computacional	Escalable solo con datos lineales	[5]
GMM	Probabilístico	Modela distribuciones mixtas	Requiere estimación precisa de parámetros	[6]
Árboles de Decisión	Supervisado	Alta interpretabilidad	Sensibles a ruido en datos	[7], [36]
AE	No supervisado	Captura patrones complejos; manejo de ruido	Sensible a sobreajuste si arquitectura es mala	[24], [25], [27], [28]
VAE y β -VAE	No supervisado	Regulariza espacio latente; buen rendimiento en datos secuenciales	Más difícil de entrenar; requiere mayor tuning	[2], [34], [35], [37]
AE + CNN (CAE)	No supervisado	Bueno para datos espaciales/visuales	Requiere preprocesamiento específico	[29]
AE + LSTM	No supervisado	Captura dependencias temporales y contextuales clave en UEBA	Elevada complejidad computacional; requiere secuencias bien definidas	[34], [38]

Tabla 3.1: Comparativa de métodos de detección de anomalías

PROPUESTA DE SOLUCIÓN Y GRADO DE INNOVACIÓN

La propuesta de solución se basa en el uso de un β -VAE para la detección de anomalías. Este enfoque se inspira en modelos generativos que utilizan un espacio latente probabilístico para representar las características subyacentes de los datos. A diferencia de los autoencoders clásicos, el β -VAE no sólo aprende a reconstruir las observaciones, sino que también modela explícitamente la incertidumbre mediante una distribución latente. Esta propiedad permite no sólo detectar anomalías a partir del error de reconstrucción, sino también cuantificar la incertidumbre asociada a dicha reconstrucción, tanto a nivel global como por característica. De este modo, es posible identificar no sólo que una observación es anómala, sino también en qué atributos específicos el modelo presenta mayor incertidumbre o error, lo que proporciona una mayor capacidad explicativa sobre el origen potencial de las anomalías. Cabe señalar que la arquitectura concreta utilizada para este modelo —descrita más abajo y empleada en los experimentos— representa una configuración posible del β -VAE, pero sus componentes (número de capas, unidades, activaciones, regularizaciones) son flexibles y pueden adaptarse a distintos dominios y objetivos. Un estudio más pormenorizado del fundamento teórico que sustenta este enfoque se presenta en [Apéndice A](#).

4.1 ARQUITECTURA DEL MODELO

La arquitectura propuesta del modelo β -VAE se muestra en la [Figura 4.1](#). El modelo sigue la clásica estructura encoder–espacio latente–decoder, pero incorpora un componente probabilístico clave en el espacio latente que permite capturar y modelar la incertidumbre inherente a los datos.

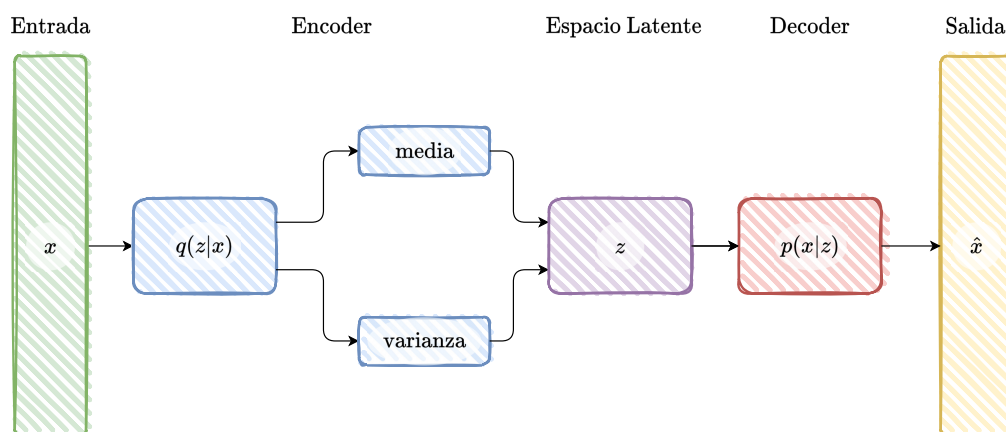


Figura 4.1: Esquema general de la arquitectura del β -VAE con sus componentes principales: encoder, espacio latente probabilístico y decoder.

Los datos de entrada (verdes) son procesados inicialmente por el **encoder** (azul), que consiste en una serie de capas densas con funciones de activación ReLU [39, 40, 41], acompañadas de normalización por lotes (*Batch Normalization* [42]) para estabilizar y acelerar el entrenamiento. Este encoder no solo codifica la información, sino que estima los parámetros de una distribución gaussiana multivariada (media μ y varianza σ^2 por dimensión) para cada instancia. La suposición de independencia entre dimensiones —implícita en la covarianza diagonal— facilita la optimización y simplifica el modelo.

A partir de esta distribución parametrizada, se realiza un muestreo estocástico que genera una representación latente (violeta). Esta muestra latente es luego pasada al **decoder** (rojo), que intenta reconstruir la entrada original, produciendo una salida (amarilla) con la misma dimensionalidad que los datos iniciales. Este proceso probabilístico permite al modelo no solo reconstruir sino también expresar la incertidumbre en la reconstrucción, fundamental para detectar anomalías con una explicación detallada.

El **encoder** (ver Figura 4.2) procesa los datos de entrada a través de una serie de capas densas con funciones de activación ReLU, acompañadas de normalización por lotes (*Batch Normalization*). Esta última estabiliza y acelera el entrenamiento al normalizar la salida de cada capa oculta, y se aplica exclusivamente en el encoder. Las activaciones ReLU introducen no linealidad sin saturación, lo que mejora la eficiencia computacional y mitiga el problema del gradiente desvanecido [43]. Al final del encoder, se generan dos salidas: el vector de medias μ y el de log-varianzas $\log \sigma^2$, que parametrizan una distribución normal multivariada con covarianza diagonal. Esta suposición implica independencia entre las dimensiones del espacio latente, lo que simplifica la estimación de parámetros al evitar el aprendizaje de correlaciones cruzadas entre variables latentes.

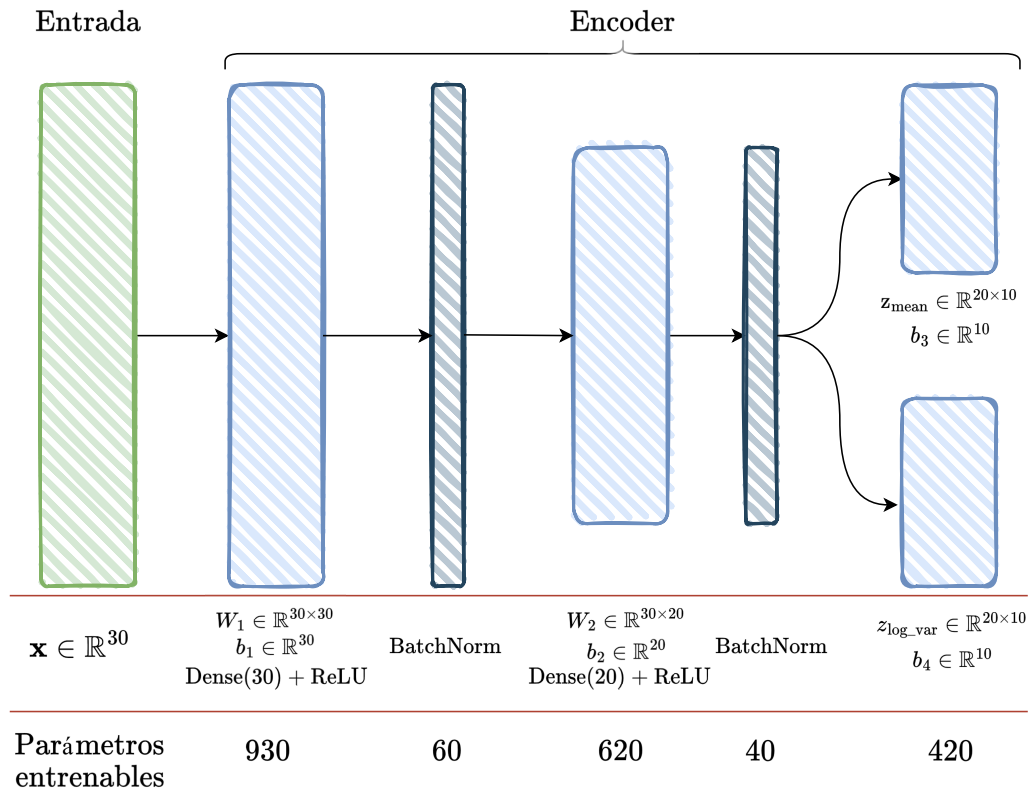


Figura 4.2: Arquitectura del encoder del modelo β -VAE. Se observa un flujo de procesamiento basado en capas densas, con normalización por lotes y funciones de activación ReLU en las capas ocultas. Las salidas z_{mean} y $z_{\text{log_var}}$, que parametrizan la distribución latente, no incorporan función de activación. Las dimensiones corresponden al modelo entrenado sobre los datos utilizados en los experimentos.

A partir de esta distribución se realiza un muestreo estocástico mediante el truco de reparametrización [3], lo cual permite la propagación del gradiente durante el entrenamiento. El vector resultante, z , representa una codificación latente de los datos y es utilizado por el **decoder** (ver Figura 4.3), que aplica una arquitectura simétrica para reconstruir la entrada original. En el **decoder** no se emplean técnicas como **Batch Normalization** ni **Dropout**. Esta decisión busca permitir que el decoder aprenda libremente la estructura de los datos a partir del espacio latente, sin introducir restricciones adicionales que puedan interferir en el proceso de reconstrucción.

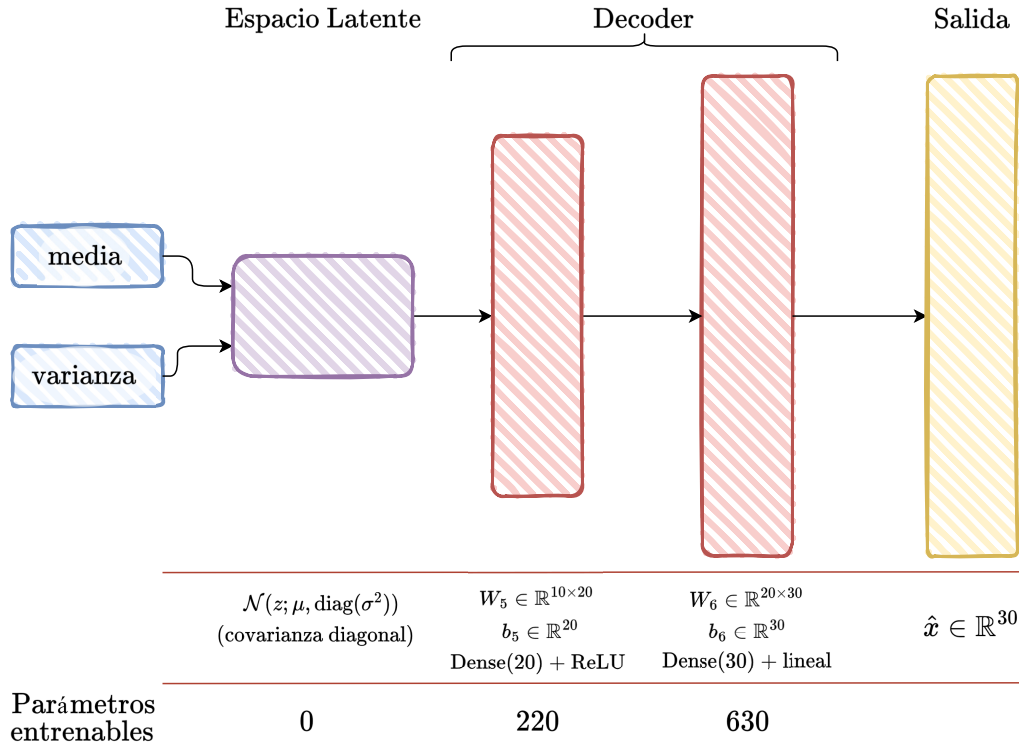


Figura 4.3: Arquitectura del decoder del modelo β -VAE. Se presenta un flujo de procesamiento simétrico al encoder, con capas densas y activaciones ReLU en las capas ocultas. La capa de salida emplea activación lineal para permitir una reconstrucción sin restricciones artificiales. Las dimensiones corresponden al modelo entrenado sobre los datos utilizados en los experimentos.

Se emplea la activación **ReLU** en todas las capas ocultas del modelo, con excepción de las capas que definen la distribución latente, y una función de activación **lineal** en la capa de salida del decoder. Esta última elección permite que los valores reconstruidos no estén restringidos artificialmente, lo que resulta particularmente útil para la detección de anomalías, ya que los outliers pueden manifestarse como desviaciones amplificadas respecto a los patrones normales.

Tanto el encoder como el decoder incluyen regularización **L2** sobre los pesos, como estrategia para prevenir el sobreajuste y mejorar la capacidad de generalización. Adicionalmente, se contempla la posibilidad de aplicar **Dropout** en las capas ocultas como mecanismo opcional de regularización, configurable según el contexto del experimento.

Se emplean distintos esquemas de inicialización de pesos con el objetivo de favorecer una propagación estable de los gradientes durante el entrenamiento, adaptándose al tipo de activación presente en cada capa. Las capas con activación ReLU utilizan la inicialización **He Normal**, adecuada para mantener la varianza a lo largo de las capas profundas. La capa que genera el vector μ (z_{mean}) adopta la inicialización **Glorot Uniform**, que promueve

un balance adecuado entre entradas y salidas en redes sin saturación. Por su parte, la capa que define $\log \sigma^2$ ($z_{\log var}$) emplea la inicialización **LeCun Normal**, apropiada para estabilizar la convergencia en variables de naturaleza logarítmica. Se ofrecen detalles adicionales sobre las propiedades y justificación de estas estrategias en el [Apéndice B](#).

A modo de síntesis estructural, la [Tabla 4.1](#) presenta la arquitectura del modelo β -VAE utilizado en los experimentos. Este modelo transforma un vector de entrada de dimensión $n_{features} = 30$ en una representación latente de dimensión 10, cuya distribución se regulariza mediante la divergencia de Kullback-Leibler con respecto a una normal estándar $\mathcal{N}(0, I)$. El decoder se encarga de reconstruir la entrada original a partir de esta representación comprimida, permitiendo tanto la evaluación de la capacidad reconstructiva del modelo como la generación de nuevas muestras plausibles dentro del dominio. Cabe destacar que se incorpora regularización **L2** en las capas del encoder, con un coeficiente $\lambda^* = 10^{-3}$, lo que contribuye a mejorar la capacidad de generalización y reducir el sobreajuste.

Tabla 4.1: Configuración del β -VAE empleado. Regularización L2: $\lambda^* = 10^{-3}$.

Bloque	Capa	Unidades / Dim.	Activación	Inicialización	Reg.	Observaciones
Encoder	Entrada	Dim = $n_{features}$	-	-	-	Vector de características original
	Dense 1	30	ReLU	He Normal	L2 (λ^*)	Primera capa oculta
	BatchNorm	-	-	-	-	Normaliza activaciones intermedias
	Dense 2	20	ReLU	He Normal	L2 (λ^*)	Segunda capa oculta
	BatchNorm	-	-	-	-	Mejora la estabilidad del entrenamiento
Latente	z_{mean}	10	Lineal	Glorot Uniform	-	Media de la distribución latente
	$z_{\log var}$	10	Lineal	LeCun Normal	-	Log-varianza (diag. de matriz de covarianza)
	Sampling	10	-	-	-	$z = \mu + \sigma \cdot \varepsilon$, con $\varepsilon \sim \mathcal{N}(0, I)$
Decoder	Dense 1	20	ReLU	He Normal	-	Primera capa del decodificador
	Dense 2	30	ReLU	He Normal	-	Segunda capa del decodificador
	Output	$n_{features}$	Lineal	-	-	Reconstrucción directa

4.2 ESPACIO LATENTE

El espacio latente en β -VAE constituye una representación compacta y continua de las observaciones originales, en la que cada punto codifica características semánticas relevantes de los datos. A diferencia de un autoencoder clásico, donde cada entrada es mapeada a un único punto en este espacio, el β -VAE modela la codificación como una distribución gaussiana multivariada. Esto implica que cada observación x_i no se codifica en un único vector latente, sino en una distribución definida por una media μ_i y una desviación estándar σ_i .

Durante el entrenamiento y la inferencia, se utiliza una variable auxiliar $\varepsilon \sim \mathcal{N}(0, 1)$ para muestrear un punto z_i en el espacio latente, utilizando la operación de *reparametrización*: $z_i = \mu_i + \sigma_i \cdot \varepsilon$. Esta técnica permite que el muestreo sea diferenciable, lo que es esencial para aplicar retropropagación.

La [Figura 4.4](#) ilustra este proceso para una observación individual x_1 , suponiendo un espacio latente de dimensión 2. A través del encoder, se obtiene una codificación probabilística (μ_1, σ_1) , que luego se convierte en un punto latente z_1 mediante la muestra con ruido gaussiano. Este punto latente representa a la observación original dentro de una estructura regularizada y continua, y es posteriormente utilizado por el decoder para obtener la reconstrucción $\hat{x}_1$.

Este espacio latente tiene varias propiedades deseables: permite una interpolación suave entre observaciones, proporciona una noción de similitud estadística entre puntos, y es regularizado para aproximarse a una distribución normal estándar. Como resultado, facilita la detección de anomalías, ya que las observaciones fuera de la distribución aprendida (con baja probabilidad latente o alto error de reconstrucción) pueden ser interpretadas como datos atípicos.

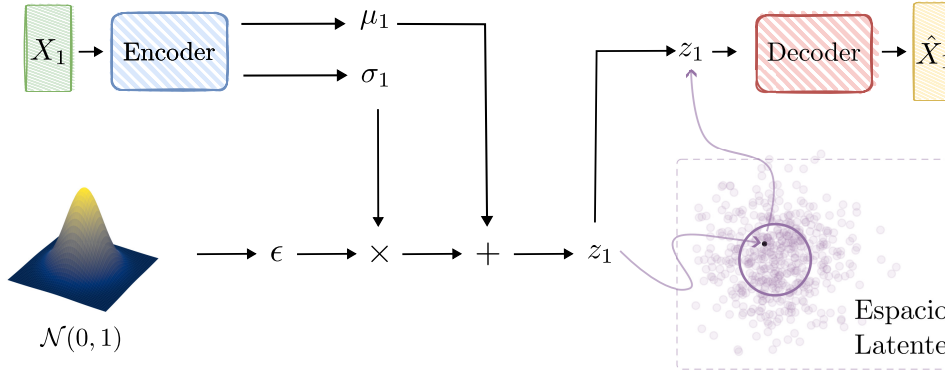


Figura 4.4: Representación esquemática del proceso de codificación y decodificación en el β -VAE. Una observación de entrada X_1 se transforma mediante el encoder en una distribución latente caracterizada por μ_1 y σ_1 . A partir de esta distribución, se realiza una muestra z_1 utilizando una variable auxiliar $\epsilon \sim \mathcal{N}(0, 1)$. Esta muestra representa a X_1 en el espacio latente y es utilizada por el decoder para reconstruir la observación como $\hat{X}_1$.

4.3 FUNCIÓN DE PÉRDIDA

Como se mencionó previamente, el β -VAE es una extensión del autoencoder variacional que introduce un mecanismo explícito para regular el equilibrio entre fidelidad en la reconstrucción y estructura en el espacio latente. En esta sección se profundiza en la función de pérdida que define su comportamiento.

La función de pérdida en un β -VAE está compuesta por dos términos: la pérdida de reconstrucción y la divergencia de Kullback-Leibler (KL). La pérdida de reconstrucción evalúa qué tan bien el modelo es capaz de reproducir la entrada original x a partir de su representación latente z , mientras que la divergencia KL actúa como un regularizador que mide cuán diferente es la distribución posterior $q(z|x)$ respecto a una distribución normal estándar $p(z) = \mathcal{N}(0, I)$.

De forma intuitiva, puede pensarse en la divergencia KL como una fuerza que “tira” de las representaciones latentes hacia una forma gaussiana idealizada, lo que facilita la continuidad, la interpolación y la generación de nuevas muestras. Sin embargo, esta fuerza compite con el objetivo de reconstrucción, que tiende a empujar las representaciones latentes hacia formas más adaptadas a cada muestra individual. La interacción entre ambos objetivos define el comportamiento del modelo y la calidad de la representación aprendida. Para más detalles matemáticos, se remite al [Apéndice A](#).

4.3.1 Interpretación del factor β

El parámetro $\beta > 0$ modula explícitamente la importancia relativa de la divergencia KL dentro de la función de pérdida total del modelo:

$$\mathcal{L}(x) = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \beta \cdot D_{\text{KL}}(q(z|x) \parallel p(z)) \quad (4.1)$$

Este parámetro permite controlar el equilibrio entre la calidad de la reconstrucción y la regularización del espacio latente. Intuitivamente:

- Si $\beta > 1$, la presión para que las distribuciones latentes $q(z|x)$ se asemejen a la distribución prior estándar $p(z) = \mathcal{N}(0, I)$ es mayor. Esto promueve un espacio latente más ordenado, compacto y estructurado, aunque a costa de perder precisión en la reconstrucción.
- Si $\beta < 1$, el modelo relaja esta presión y se concentra más en reconstruir fielmente las entradas, lo que puede derivar en un espacio latente más disperso y adaptado específicamente a los datos, pero menos útil para generalizar o generar nuevas muestras. Cuando β es muy pequeño, la función de pérdida se aproxima más a la del Autoencoder clásico, pues la penalización por desviación de la distribución latente respecto a la normal estándar es mínima, lo que permite que el espacio latente se adapte libremente a los datos, perdiendo la estructura probabilística propia del VAE.

De forma gráfica, como puede observarse en la [Figura 4.5](#), puede interpretarse que:

- Con β alto, el modelo prioriza la regularización del espacio latente, generando representaciones latentes más similares entre sí y cercanas al origen, sacrificando detalles en las reconstrucciones.
- Con β bajo, el modelo prioriza la fidelidad en la reconstrucción, permitiendo mayor diversidad y separación entre las representaciones latentes, a costa de perder la estructura regular del espacio latente.

Este factor cumple además un rol regularizador fundamental: al controlar la varianza y forma de las distribuciones latentes, se evita el sobreajuste y se favorece una codificación más robusta y generalizable.

4.4 TÉCNICAS DE ESCALADO DE CARACTERÍSTICAS

La transformación de las características numéricas es un paso esencial previo al entrenamiento de modelos basados en redes neuronales, especialmente cuando se requiere preservar relaciones relativas entre las variables o minimizar el impacto de valores extremos.

- **Estandarización (Z-score):** transforma cada variable para que tenga media cero y desviación estándar uno. Esta técnica es sensible a valores atípicos, por lo que puede resultar subóptima si se espera la presencia de anomalías dentro del conjunto de entrenamiento. Dado un valor x , la estandarización se define como:

$$z = \frac{x - \mu}{\sigma} \quad (4.2)$$

donde μ es la media y σ la desviación estándar de la variable.

- **Normalización Min-Max:** ajusta los valores para que se ubiquen en un rango fijo (usualmente $[0, 1]$). Es útil cuando se desea preservar la forma de la distribución relativa entre las variables, pero también puede ser sensible a outliers. Se calcula como:

$$x_{\text{norm}} = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}} \quad (4.3)$$

donde $x_{\text{mín}}$ y $x_{\text{máx}}$ son los valores mínimo y máximo observados, respectivamente.

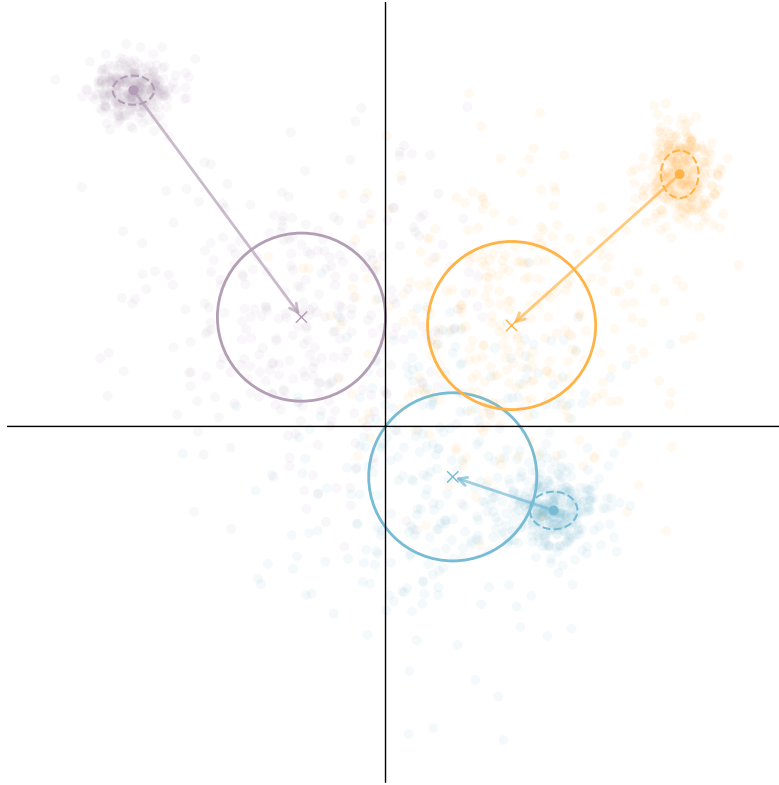


Figura 4.5: Representación del efecto del parámetro β en la dispersión y centro de las distribuciones latentes. Las flechas indican el desplazamiento del centroide de cada clase al aumentar β , mientras que las elipses muestran la forma y extensión de la varianza aprendida. Los ejes corresponden a las dos primeras componentes principales del espacio latente, lo que permite visualizar su estructura en un plano reducido.

- **Transformación robusta (RobustScaler):** utiliza la mediana y el rango intercuartílico (IQR) en lugar de la media y desviación estándar. Es especialmente útil en contextos no supervisados, donde se espera una pequeña proporción de datos anómalos incrustados dentro del conjunto de entrenamiento. Esta técnica reduce el efecto de los outliers sin requerir conocimiento previo sobre qué datos son anómalos. La transformación se expresa como:

$$x_{\text{rob}} = \frac{x - \text{mediana}(x)}{\text{IQR}(x)} \quad (4.4)$$

donde $\text{IQR}(x) = Q_3 - Q_1$, siendo Q_1 y Q_3 los cuartiles primero y tercero, respectivamente.

En este trabajo, se empleó la estandarización mediante Z-score para el entrenamiento del modelo. Sin embargo, se deja planteada como línea futura la evaluación comparativa con transformaciones robustas en escenarios donde los datos normales no puedan ser claramente separados de los anómalos.

4.5 ENTRENAMIENTO DEL MODELO

El entrenamiento del modelo implicó la selección y ajuste de diversos hiperparámetros clave, que influyen directamente en su capacidad para aprender representaciones útiles y generalizables. Los principales hiperparámetros considerados fueron:

- **Dimensión del espacio latente** (*latent dimension*): tamaño del espacio donde se proyectan las representaciones comprimidas.
- **Dimensiones de las capas ocultas** del encoder y decoder: definen la capacidad del modelo para capturar relaciones no lineales en los datos.
- **Parámetro β** de regularización del término Kullback-Leibler, característico del β -VAE, que controla el balance entre reconstrucción y regularización.
- **Tasa de aprendizaje** (*learning rate*): determina el tamaño de los pasos en la actualización de pesos durante el descenso del gradiente. Se empleó el optimizador *Adam*, reconocido por su eficiencia en modelos no lineales y adaptabilidad.
- **Tamaño del minibatch** (*batch size*): número de muestras procesadas en cada iteración del entrenamiento.
- **Parámetro λ** del regularizador L2: penaliza la magnitud de los pesos para evitar sobreajuste, contribuyendo a una mejor generalización.

Se definió una arquitectura del encoder con dos capas ocultas de dimensiones 30 y 20, respectivamente, que van disminuyendo progresivamente hacia el espacio latente. El decoder replicó esta estructura de manera simétrica. Para el regularizador L2 se empleó un valor fijo de $\lambda = 0.001$, que mostró buen desempeño en pruebas preliminares y se mantuvo constante durante el resto del proceso. Para un análisis más detallado de los resultados obtenidos, puede consultarse la [Sección 5.5](#).

Respecto a la tasa de aprendizaje, se exploraron distintas combinaciones con otros hiperparámetros y se determinó que un valor de 0.001 proporcionaba un adecuado balance entre rapidez de convergencia y estabilidad en el entrenamiento.

La optimización de los demás hiperparámetros, como la dimensión del espacio latente y el parámetro β , se llevó a cabo inicialmente mediante una búsqueda aleatoria (*random search*) dentro de un espacio razonable de configuraciones. Posteriormente, se aplicó una búsqueda en malla (*grid search*) más focalizada sobre los resultados prometedores, lo que permitió refinar la selección final.

Además, se implementó una estrategia de *early stopping* para prevenir el sobreajuste y optimizar los recursos computacionales. Se configuró con una paciencia de 10 épocas y un umbral mínimo de mejora de 0.01 en la pérdida de validación. De esta forma, el entrenamiento se detuvo automáticamente cuando no se observaron mejoras significativas en la función objetivo durante varias iteraciones consecutivas, garantizando un buen compromiso entre ajuste y generalización.

Como parte del proceso, una vez entrenado el modelo, se calibró un umbral sobre el error cuadrático medio (MSE) de reconstrucción utilizando un pequeño subconjunto de datos de validación. Este umbral se ajustó para que aproximadamente el 1 % de las muestras quedaran por encima del mismo, definiendo así el límite para clasificar observaciones como outliers. Finalmente, se evaluó el desempeño del modelo y la configuración de hiperparámetros seleccionados sobre el conjunto de test, que permaneció totalmente separado del entrenamiento y calibración, asegurando una evaluación objetiva.

4.6 EVALUACIÓN DEL MODELO

Para cuantificar el desempeño del modelo se emplearon las siguientes métricas:

- **Error Cuadrático Medio (MSE)** de reconstrucción: mide la precisión del modelo al reconstruir las muestras normales.

- **Precisión, Recall y F1-score:** calculadas con base en la clasificación binaria obtenida utilizando el umbral calibrado sobre el MSE para detectar anomalías.

Dado el marcado desbalance entre muestras normales y anómalas, no se consideró la curva ROC como métrica principal, ya que puede resultar menos representativa en estos contextos. En cambio, las métricas de Precisión, Recall y F1-score ofrecen una evaluación más adecuada, enfocándose en el desempeño del modelo para identificar correctamente la clase minoritaria (anomalías).

Estas métricas se calculan a partir de la matriz de confusión, que resume las cantidades de predicciones correctas e incorrectas clasificadas como verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos:

		Predicción	
		Normal 0	Anómala 1
Real	Normal 0	TN (Verdaderos Negativos)	FP (Falsos Positivos)
	Anómala 1	FN (Falsos Negativos)	TP (Verdaderos Positivos)

Tabla 4.2: Matriz de Confusión para la detección binaria de anomalías

A partir de esta matriz, se definen las métricas clave como:

$$\text{Precisión} = \frac{TP}{TP + FP} \quad , \quad \text{Recall} = \frac{TP}{TP + FN} \quad , \quad F1\text{-score} = 2 \cdot \frac{\text{Precisión} \times \text{Recall}}{\text{Precisión} + \text{Recall}}$$

- **Precisión:** mide la proporción de muestras clasificadas como anómalas que son realmente anómalas, es decir, qué tan confiables son las predicciones positivas.
- **Recall (Sensibilidad):** mide la proporción de anomalías reales que fueron correctamente detectadas, reflejando la capacidad del modelo para encontrar todos los casos anómalos.
- **F1-score:** es la media armónica entre Precisión y Recall, balanceando ambas métricas en un solo valor, especialmente útil cuando las clases están desbalanceadas.

Estas métricas fueron calculadas utilizando el conjunto de test, que no se empleó ni en el entrenamiento ni en la calibración del umbral, garantizando así la validez e imparcialidad de la evaluación.

4.6.1 Interpretación de las Métricas de Evaluación y del Umbral

En la detección de anomalías, el umbral que separa las observaciones normales de las anómalas puede definirse tanto sobre el **Error Cuadrático Medio (MSE)** como sobre su raíz cuadrada, el **Root Mean Squared Error (RMSE)**. Matemáticamente, establecer un umbral sobre el MSE o sobre el RMSE es equivalente, ya que uno es la raíz cuadrada del otro. Sin embargo, desde el punto de vista interpretativo, el RMSE resulta más intuitivo porque se expresa en las mismas unidades que los datos originales (o en unidades de desviación estándar si los datos fueron estandarizados), mientras que el MSE está en unidades al cuadrado.

Formalmente, el MSE se define como:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2, \quad (4.5)$$

y el RMSE es su raíz cuadrada:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2}. \quad (4.6)$$

En este trabajo, los datos de entrada fueron **estandarizados** mediante la transformación Z-score, es decir, cada variable fue escalada para tener media cero y varianza uno. Esta estandarización es fundamental para el correcto funcionamiento del modelo, ya que normaliza la escala de las variables y facilita el aprendizaje.

Además, trabajar con datos estandarizados permite que el RMSE tenga una interpretación estadística directa y clara: representa la *desviación promedio* entre el valor original x_i y su reconstrucción $\hat{x}_i$, expresada en unidades de desviación estándar. Por ejemplo, un RMSE de 0.5 indica que, en promedio, la reconstrucción se desvía medio desvío estándar respecto al valor real.

La elección de la estandarización Z-score es indispensable en este contexto por varias razones. Primero, otras técnicas de normalización, como la *Min-Max scaling*, no son adecuadas cuando existen outliers en los datos, ya que los valores extremos comprimen la mayoría de los datos normales en un rango muy pequeño cercano a cero. Esto dificulta la diferenciación entre observaciones normales y anómalas y entorpece el entrenamiento del modelo. Segundo, la estandarización contribuye a evitar problemas comunes en redes neuronales, como el *desvanecimiento del gradiente*, que puede ocurrir si la desviación estándar de las variables es muy baja, limitando la magnitud de las actualizaciones durante el entrenamiento.

Finalmente, para la detección de anomalías, se define un umbral τ sobre el RMSE (o MSE), de modo que cualquier observación cuyo error de reconstrucción supere este valor se clasifica como anómala:

$$\text{Anomalía si } \text{RMSE}(x) > \tau. \quad (4.7)$$

Este umbral se determina de forma empírica a partir de un conjunto de validación independiente, fijando un percentil alto (por ejemplo, el percentil 99) sobre la distribución del RMSE para las muestras normales. Esta estrategia permite controlar la tasa esperada de falsos positivos, bajo la suposición de que los errores de reconstrucción de las muestras normales se distribuyen dentro de un rango definido.

RESULTADOS Y ANÁLISIS

5.1 DESCRIPCIÓN DEL CONJUNTO DE DATOS

El conjunto de datos utilizado proviene de [44]. Contiene un total de 284,807 transacciones de tarjeta de crédito, de las cuales 492 son fraudulentas (alrededor del 0.172 %). Las variables han sido transformadas mediante Análisis de Componentes Principales (PCA) para preservar la confidencialidad de los datos originales, con excepción de las variables Amount y Time. El conjunto final incluye 30 características numéricas: 28 componentes principales (V1 a V28), junto a las dos variables no transformadas.

Las clases están severamente desbalanceadas, una característica común en problemas de detección de fraude. En el [Apéndice C](#), se incluyen gráficos adicionales con las distribuciones de las variables y una descripción estadística más detallada, lo cual permite al lector tener un panorama más completo del conjunto de datos.

5.2 PREPROCESAMIENTO DE LOS DATOS

Los datos fueron estandarizados utilizando la transformación *Z-score*, ya que la varianza de las características presentaba valores inferiores a 0,007. Esta elección se realizó con el objetivo de preservar la información relativa sin comprimir artificialmente la escala de los datos. La estandarización se aplicó únicamente sobre el conjunto de entrenamiento, y se almacenó el objeto del escalador para garantizar una transformación coherente sobre los conjuntos de validación, prueba y predicción. Finalizado el proceso de inferencia, es posible revertir esta transformación mediante la inversa del escalador aplicado.

5.3 ENTRENAMIENTO DEL MODELO

Se utilizó únicamente la clase normal (`Class = 0`) para entrenar y validar el modelo, siguiendo el enfoque típico de entrenamiento no supervisado en detección de anomalías. Esta decisión se justifica en parte porque, en escenarios reales de ciberseguridad —como en el análisis de registros de eventos (*logs*)—, no se dispone de etiquetas explícitas que clasifiquen cada observación como normal o anómala. Por ello, es común asumir que los eventos anómalos son minoritarios dentro del conjunto y que los datos disponibles corresponden en su mayoría a comportamientos normales.

En este contexto, resulta fundamental aplicar técnicas de escalado adecuadas que reduzcan la influencia de posibles valores atípicos durante el entrenamiento. Además de la estandarización clásica mediante *Z-score*, se

pueden emplear enfoques más robustos como *RobustScaler*, que preservan mejor la estructura general del conjunto de datos al ser menos sensibles a outliers. Estas estrategias se desarrollan con mayor detalle en la [Sección 4.4](#).

En la [Figura 5.1](#) se muestra la evolución de la función de pérdida durante el entrenamiento, la cual permite evaluar la estabilidad y convergencia del modelo.

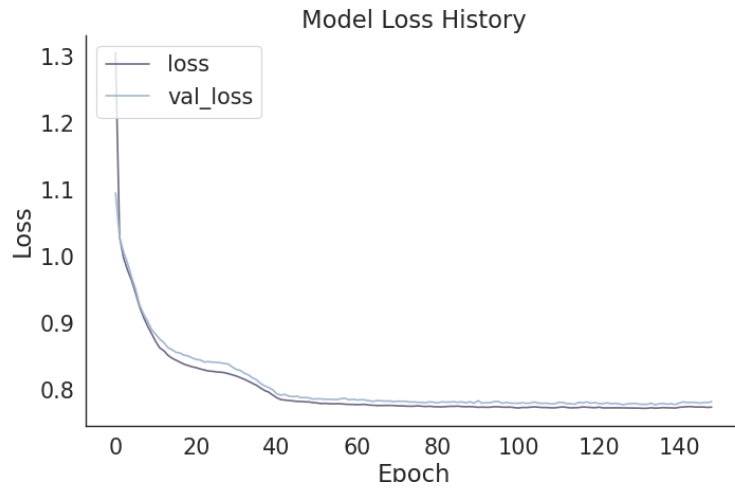


Figura 5.1: Evolución de las funciones de pérdida durante el entrenamiento.

5.4 SELECCIÓN DEL UMBRAL DE RECONSTRUCCIÓN

A partir del conjunto de validación (compuesto por datos normales), se determinó un umbral de error de reconstrucción basado en el percentil 99. Este valor se utiliza como límite superior para distinguir entre observaciones normales y potenciales anomalías. Este enfoque resulta especialmente útil en contextos donde no se cuenta con etiquetas claras que identifiquen cada observación como normal o anómala, como ocurre frecuentemente en entornos de registros de actividad (*logs*), donde los datos etiquetados son escasos o inexistentes. En tales casos, asumir que las anomalías conforman una proporción ínfima del total permite aplicar métodos basados en percentiles sin requerir supervisión explícita.

En la [Figura 5.2](#) se muestra la distribución del error de reconstrucción por característica sobre el conjunto de prueba, destacando los umbrales correspondientes a los percentiles 95 (en rojo) y 99 (en azul). Cabe aclarar que tanto esta figura como la [Figura 5.3](#) provienen de una simulación cuyo único propósito es ilustrar visualmente estos comportamientos, y fueron generadas utilizando un conjunto distinto de hiperparámetros y eliminando la variable *Time* de las características.

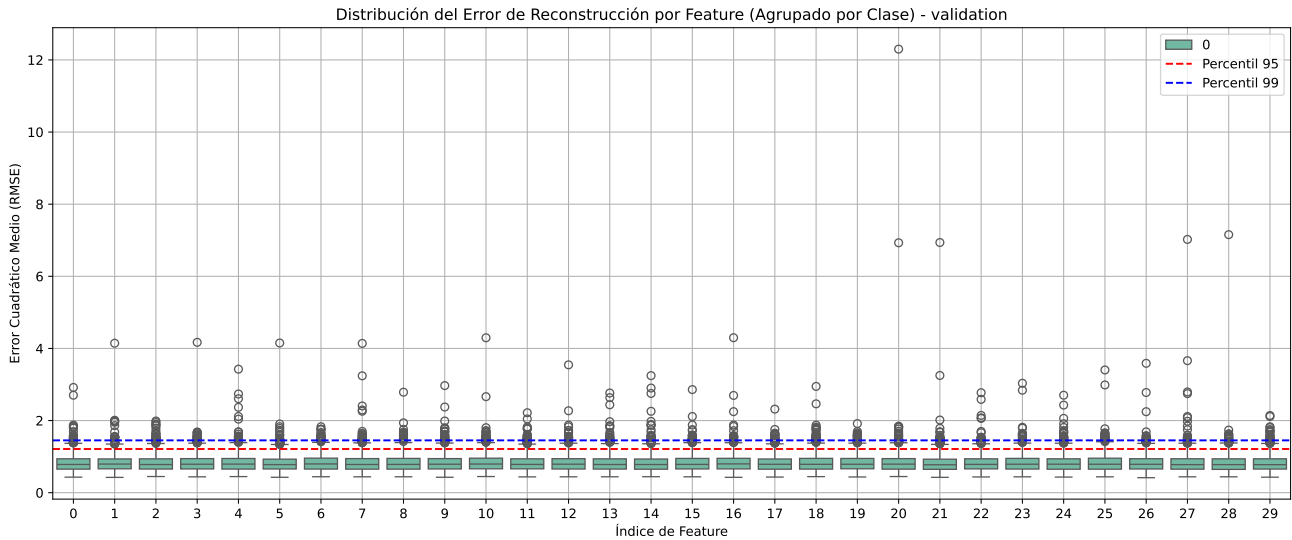


Figura 5.2: Distribución del error de reconstrucción por característica en el conjunto de validación, utilizada para analizar la variabilidad del error y seleccionar un umbral adecuado de detección. Se comparan dos posibles thresholds, correspondientes a los percentiles 95 (línea roja) y 99 (línea azul). Los puntos negros representan valores que exceden los límites del rango intercuartílico y destacan casos con errores de reconstrucción significativamente elevados.

5.4.1 Evaluación sobre el conjunto de prueba

El conjunto de prueba incluye tanto transacciones normales como fraudulentas. Se aplicó el umbral definido en la sección anterior y se evaluó el desempeño del modelo. La [Figura 5.3](#) muestra cómo varía el error de reconstrucción por característica entre clases normales y anómalas.

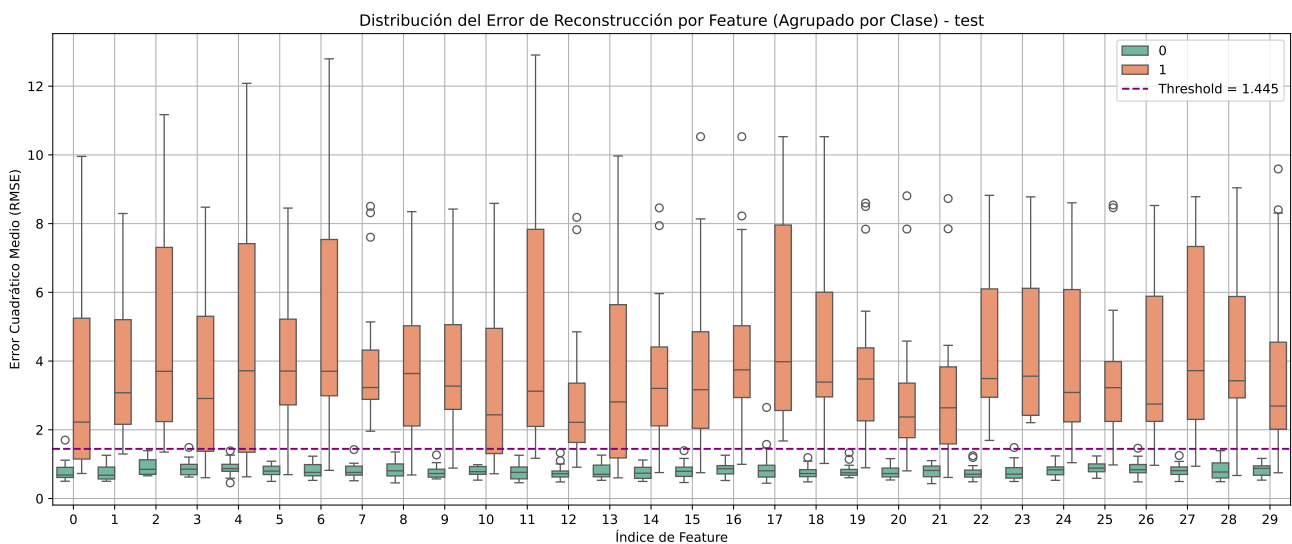


Figura 5.3: Error de reconstrucción por característica en test, separando por clase (0: normal, 1: anomalía) y utilizando el umbral obtenido previamente tomando el percentil 99. Los puntos negros representan valores que exceden los límites del rango intercuartílico.

5.5 BÚSQUEDA DE HIPERPARÁMETROS: GRID SEARCH Y RANDOM SEARCH

Con el objetivo de identificar la combinación de hiperparámetros que mejor se adapte a las características del conjunto de datos, se llevaron a cabo dos búsquedas aleatorias (*Random Search*) y una búsqueda exhaustiva (*Grid Search*). En todos los casos, el criterio principal de selección fue maximizar el valor del F1-score, al considerar tanto la precisión como el *recall*, métricas especialmente relevantes en escenarios de detección de anomalías con clases desbalanceadas. Cabe destacar que esta optimización del F1-score no se realiza directamente sobre la calidad de la reconstrucción, sino sobre el resultado de entrenar modelos que minimizan una función de pérdida conjunta, compuesta por el error de reconstrucción y la divergencia Kullback-Leibler. Esto implica que el modelo no necesariamente busca reconstrucciones perfectas, sino representaciones latentes que favorezcan una separación efectiva entre datos normales y anómalos. Un análisis detallado del comportamiento de los modelos entrenados con dos combinaciones distintas del hiperparámetro β , el cual influye directamente en la estructura y dispersión del espacio latente, puede encontrarse en el [Apéndice D](#).

Los principales hiperparámetros considerados durante la búsqueda fueron la dimensión del espacio latente (*latent dimension*) y el coeficiente de regularización de Kullback-Leibler (β), dada su influencia directa en la capacidad de representación del modelo y el equilibrio entre reconstrucción y regularización.

En la primera búsqueda aleatoria, cuyos resultados se presentan en la [Figura 5.4](#), se exploró un espacio amplio de configuraciones. Posteriormente, se realizó una segunda *Random Search* más acotada, centrada en las combinaciones más prometedoras identificadas previamente.

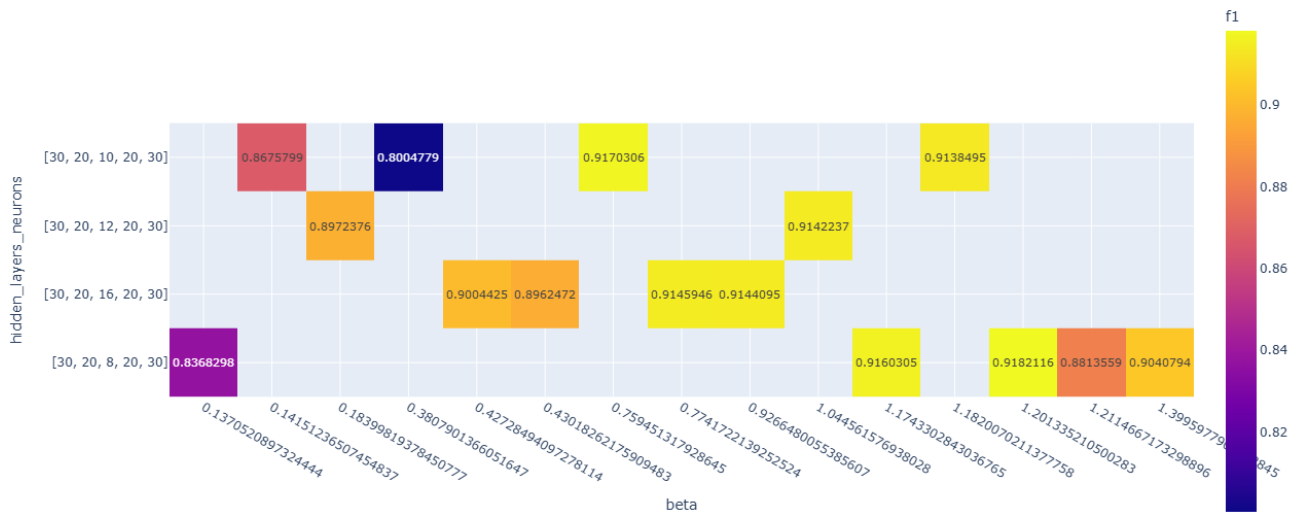


Figura 5.4: Heatmap del F1-Score obtenido mediante *Random Search* sobre combinaciones de la dimensión del espacio latente (8, 10, 12, 16) y valores aleatorios de β en el intervalo [0.13, 1.4].

A partir de los resultados obtenidos en estas búsquedas iniciales, se diseñó una *Grid Search* focalizada, cuyos resultados se muestran en la [Figura 5.5](#).

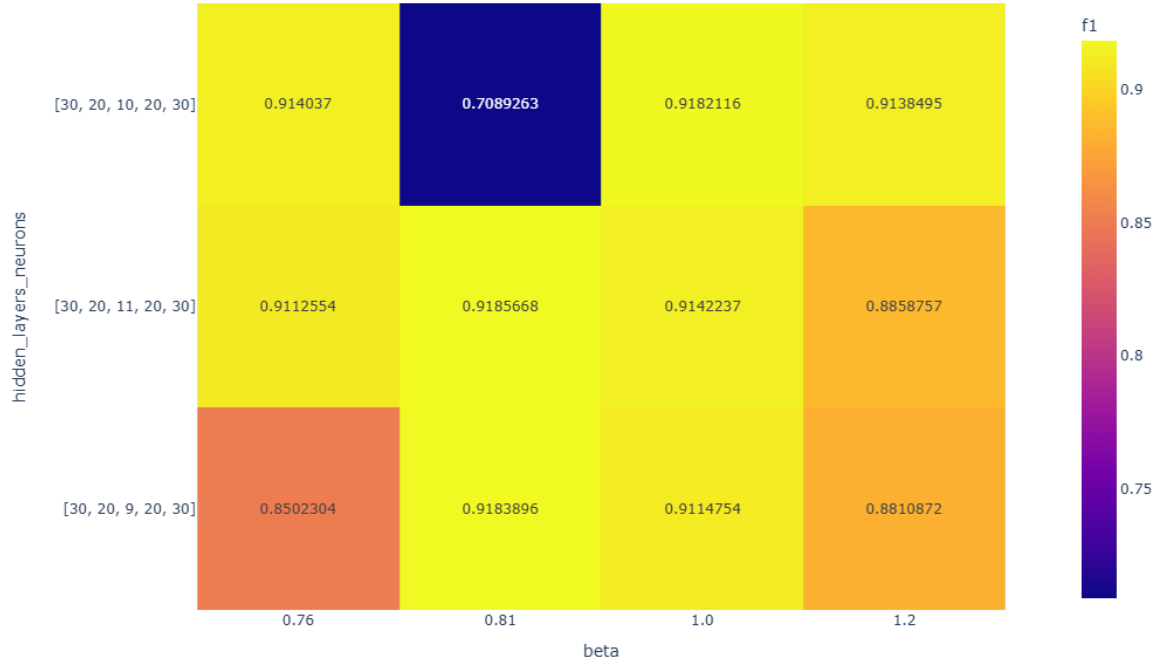


Figura 5.5: Heatmap del F1-Score obtenido mediante *Grid Search* sobre la dimensión del espacio latente (9, 10, 11) y los siguientes valores de β (0.76, 0.81, 1.0, 1.2).

La mejor configuración identificada fue la siguiente:

- $\beta = 0.81$
- Capas del codificador: [20, 30]
- Dimensión latente: 11
- Capas del decodificador: [30, 20]
- Parámetro de regularización L2: 0.001
- Tasa de aprendizaje: 0.001
- Tamaño de batch: 256
- Número máximo de épocas: 200
- *early stopping*: activado

Con esta configuración, se entrenó el modelo final, obteniéndose las métricas de evaluación que se presentan en la [Tabla 5.1](#). La matriz de confusión correspondiente se muestra en la [Figura 5.6](#), reflejando la capacidad del modelo para distinguir eficazmente entre datos normales y anómalos. En este contexto, la clase 0 representa observaciones normales (negativas) y la clase 1 corresponde a observaciones anómalas (positivas).

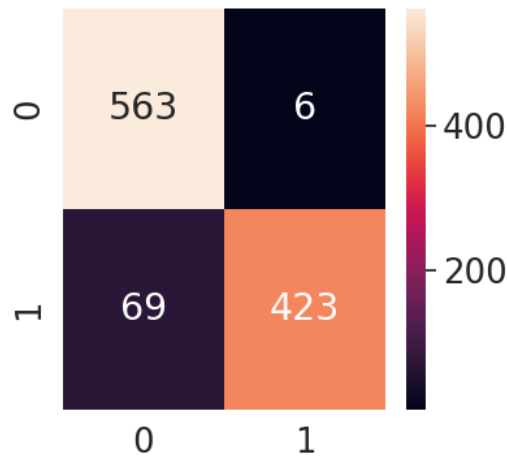


Figura 5.6: Matriz de confusión obtenida del modelo β -VAE, donde se muestran los valores de verdaderos negativos (563), falsos positivos (6), falsos negativos (69) y verdaderos positivos (423). Esta matriz refleja el desempeño del clasificador en la discriminación entre clases, indicando un alto número de aciertos y un bajo número de errores de clasificación.

El umbral reportado en la [Tabla 5.1](#) fue calculado sobre el error cuadrático medio (MSE). Por lo tanto, su raíz cuadrada $\sqrt{\text{Threshold}} = 1.4312$ puede interpretarse como una medida del error en las mismas unidades que la desviación estándar, permitiendo evaluar la magnitud de las anomalías en términos comparables a dicha métrica estadística.

Métrica	Valor
Threshold	2.0484
Tasa de verdaderos negativos (TNR)	0.9895
Tasa de falsos positivos (FPR)	0.0105
Recall (Sensibilidad)	0.8598
Accuracy (Exactitud)	0.9293
Precisión	0.9860
F1-Score	0.9186
AUC-ROC	0.9596

Tabla 5.1: Métricas de evaluación del modelo de detección de anomalías

5.6 COMPARACIÓN CON OTROS MODELOS

En esta sección se realiza una comparación entre el modelo β -VAE, el AE clásico, Isolation Forest (IF) y Extended Isolation Forest (EIF). Cabe destacar que tanto el AE clásico, como el IF y el EIF se les aplicó la transformación consistente en un *Robust Scaler* seguido de una normalización *Min-Max*. Esta secuencia de preprocesamiento tiene un impacto directo en la escala de los datos reconstruidos, lo que explica la diferencia significativa en los valores de umbral (*threshold*) entre el AE clásico, IF y EIF, y el β -VAE.

La [Tabla 5.2](#) resume las métricas de desempeño de los modelos evaluados. Se observa que el β -VAE ofrece una combinación competitiva de precisión, sensibilidad y área bajo la curva ROC (AUC-ROC), lo que lo

posiciona como una opción robusta frente a las demás técnicas evaluadas. Cabe destacar que la curva ROC es especialmente útil en este caso, dado que la distribución de muestras en el conjunto de prueba es aproximadamente equilibrada entre clases normales y anómalas, lo que permite que esta métrica refleje adecuadamente la capacidad discriminativa de los modelos. Complementariamente, las [Figura 5.7\(a\)–\(c\)](#) presentan las matrices de confusión de los modelos Autoencoder, Isolation Forest y Extended Isolation Forest, respectivamente, proporcionando una visualización detallada de los aciertos y errores cometidos por cada método en la clasificación de observaciones normales y anómalas.

Tabla 5.2: Comparación de métricas entre distintos modelos de detección de anomalías

Métrica	Autoencoder Variacional	Autoencoder Clásico	Isolation Forest	Extended Isolation Forest
Threshold	2.0484	0.0016	0.5241	0.5233
Tasa de verdaderos negativos (TNR)	0.9895	0.9895	0.9859	0.9877
Tasa de falsos positivos (FPR)	0.0105	0.0105	0.0141	0.0123
Recall (Sensibilidad)	0.8598	0.8415	0.7215	0.7602
Accuracy (Exactitud)	0.9293	0.9208	0.8633	0.8822
Precisión	0.9860	0.9857	0.9780	0.9816
F1-score	0.9186	0.9079	0.8304	0.8568
AUC-ROC	0.9596	0.9463	0.9510	0.9557

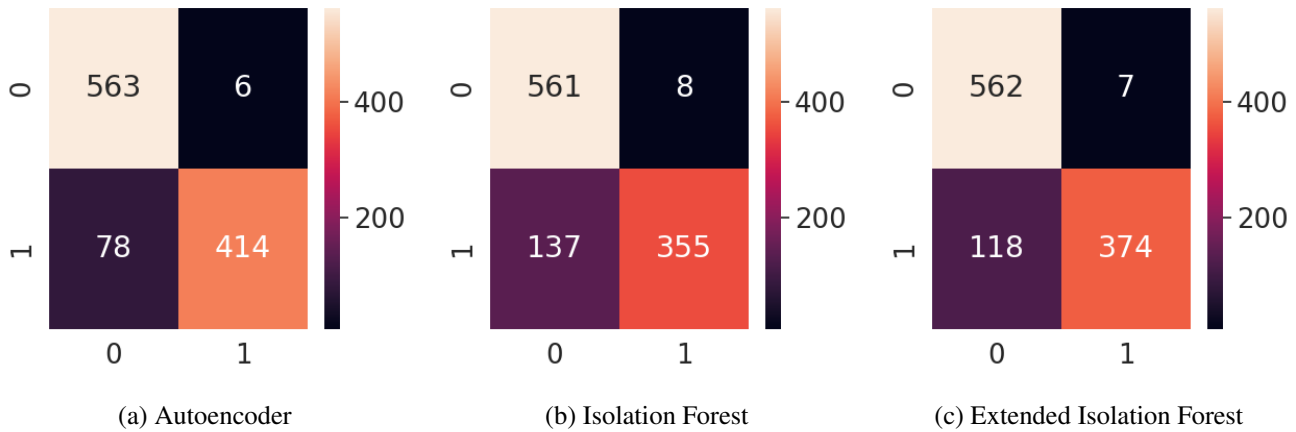


Figura 5.7: Matrices de confusión de los modelos: (a) Autoencoder: 563 TN, 6 FP, 78 FN, 414 TP; (b) Isolation Forest: 561 TN, 8 FP, 137 FN, 355 TP; (c) Extended Isolation Forest: 562 TN, 7 FP, 118 FN, 374 TP. Se observa que los modelos basados en árboles tienen mayor cantidad de falsos negativos en comparación al autoencoder.

Adicionalmente, la [Tabla 5.3](#) presenta una comparación integral de los tiempos promedio de inferencia por muestra y de entrenamiento para los modelos evaluados. La distinción entre tiempos de inferencia en datos normales y anómalos es relevante en contextos de aplicación en línea o tiempo real, donde la latencia es crítica.

Los valores de inferencia están expresados en milisegundos (ms) por muestra, mientras que los tiempos de entrenamiento se indican en horas, minutos y segundos.

Se observa una disparidad considerable en los tiempos de inferencia. Los métodos basados en árboles abarcan extremos opuestos: *Isolation Forest* (IF) destaca por su alta eficiencia con menos de 0.03 ms por muestra, mientras que su variante extendida (EIF) implica una penalización significativa, superando los 4.6 ms. El modelo *Extended Isolation Forest* (H2O) ofrece un punto intermedio con tiempos inferiores a los 2.14 ms. Por su parte, los autoencoders —tanto clásico (AE) como variacional (β -VAE)— exhiben tiempos de inferencia intermedios, entre 0.16 y 0.47 ms, siendo el β -VAE ligeramente más costoso. *

Un patrón común a todos los modelos es que las muestras anómalas se procesan más rápidamente que las normales. Esto puede explicarse porque las anomalías, al alejarse del comportamiento aprendido, generan representaciones internas menos detalladas. En los autoencoders, dicha simplificación implica menos activaciones significativas y un proceso de reconstrucción más rápido, funcionando como un “early exit” implícito. La diferencia en tiempos de inferencia alcanza hasta un 60 % en estos modelos, lo cual puede ser aprovechado para diseñar sistemas que optimicen la inferencia adaptándose dinámicamente al tipo de dato, mejorando la respuesta en escenarios críticos como la ciberseguridad.

En cuanto al entrenamiento, los tiempos reflejan diferencias marcadas pero con menor impacto en aplicaciones en línea, ya que esta fase suele ser menos frecuente. Modelos como *Isolation Forest* pueden entrenarse en segundos, mientras que el β -VAE requiere más de una hora. Este contraste refuerza la importancia de priorizar la eficiencia en la inferencia para sistemas de detección en tiempo real, sin descuidar la viabilidad operativa del reentrenamiento periódico o continuo.

Tabla 5.3: Tiempos promedio de inferencia y entrenamiento por modelo

Modelo	Tiempos promedio		
	Predicción (ms por muestra)		Entrenamiento (hh:mm:ss)
	Datos Normales	Datos Anómalos	
Autoencoder Clásico (AE)	0.394	0.165	0:05:09
Variational Autoencoder (β -VAE)	0.470	0.174	1:24:15
Isolation Forest (IF)	0.027	0.027	0:00:02
Extended Isolation Forest (EIF)	4.975	3.684	0:19:20
Extended Isolation Forest (H2O)	2.139	1.137	0:00:43

* EIF y EIF (H2O) corresponden a implementaciones del Extended Isolation Forest: la primera usa la librería eif de Python (`from eif import iForest`), mientras que la segunda emplea la plataforma H2O (`import h2o`), con diferencias en optimización y desempeño.

CONCLUSIONES Y TRABAJO A FUTURO

6.1 CONCLUSIONES

En este trabajo se desarrolló e integró con éxito un modelo de autoencoder variacional β (β -VAE) orientado a la detección de anomalías en entornos UEBA (User and Entity Behavior Analytics). Gracias a la formulación probabilística inherente al β -VAE, se logró estructurar un espacio latente informativo que facilita la separación más efectiva entre patrones normales y anómalos, superando las limitaciones de modelos discriminativos clásicos.

El modelo demostró un desempeño competitivo sobre un conjunto de datos públicos representativos de ciberseguridad, validando su capacidad para capturar patrones complejos en escenarios realistas. Se diseñó además una arquitectura modular y flexible que permite un ajuste sencillo tanto de la estructura del modelo como de sus hiperparámetros y funciones de pérdida, favoreciendo la experimentación, escalabilidad y reproducibilidad.

Un aporte destacado fue la integración progresiva del modelo dentro de una base de código amplia y profesional, construida a lo largo de más de dos años. Esta infraestructura robusta y mantenible no solo facilita el entrenamiento y evaluación offline, sino que además ya incluye la integración del modelo en pipelines de procesamiento en *streaming*, habilitando su operación en tiempo real con tecnologías Big Data como Apache Kafka y Apache Spark, consolidando así su aplicabilidad en entornos productivos.

Desde una perspectiva aplicada, la solución representa un avance concreto en la incorporación de técnicas avanzadas de aprendizaje no supervisado dentro de pipelines de detección de anomalías en ecosistemas Big Data. La inclusión de mecanismos de visualización y análisis detallado del error de reconstrucción contribuye a mejorar la interpretabilidad del modelo, un aspecto crucial para su adopción efectiva por parte de los equipos de ciberseguridad.

6.2 TRABAJO A FUTURO

Los siguientes pasos se orientan a la consolidación y extensión del modelo hacia escenarios operativos y producción, enfatizando en:

- Aplicar el modelo a datos reales de logs generados en entornos de seguridad informática, aprovechando la infraestructura ya desarrollada.
- Investigar la incorporación de umbrales personalizados por característica para mejorar la precisión de la detección, junto con una evaluación de la incertidumbre del error desde una perspectiva probabilística.
- Analizar la diferencia en la explicabilidad de los resultados entre el modelo propuesto y los enfoques clásicos (e.g., Isolation Forest, árboles, autoencoders), evaluando la trazabilidad de las decisiones anómalas

y la interpretabilidad del espacio latente generado por el VAE. Este análisis resulta clave para justificar la adopción del modelo en entornos sensibles, donde la comprensión del comportamiento anómalo es tan importante como su detección.

- Optimizar el rendimiento del sistema en entornos distribuidos, ajustando tanto el modelo como el pipeline para lograr baja latencia, alto *throughput* y eficiencia en el uso de recursos, características esenciales en entornos Big Data reales.
- Incorporar visualizaciones integradas al flujo del proyecto, incluyendo la representación del espacio latente mediante técnicas de reducción de dimensionalidad como PCA, facilitando la interpretación de las distribuciones generadas por el modelo. Complementar estas visualizaciones con ejemplos gráficos de reconstrucciones de muestras normales y anómalas, permitiendo un análisis visual de la calidad del modelo.

La plataforma desarrollada ofrece una base sólida y flexible para abordar estos retos, consolidando una solución robusta, escalable y adaptable que contribuye significativamente al avance de la detección automática de anomalías en contextos de ciberseguridad, particularmente en entornos Big Data donde la eficiencia, la interpretabilidad y la capacidad de operar en tiempo real resultan críticas.

FUNDAMENTO TEÓRICO

En este capítulo se presentan los fundamentos teóricos que sustentan el diseño y funcionamiento del VAE. Se abordarán los conceptos probabilísticos que motivan el modelo, la formulación del modelo generativo latente, la inferencia variacional amortiguada, y los métodos de optimización, incluyendo el truco de reparametrización.

A.1 FUNDAMENTOS PROBABILÍSTICOS DEL AUTOENCODER VARIACIONAL

Partimos del supuesto de que disponemos de un conjunto de datos $\mathcal{D} = \{x_i\}_{i=1}^N$ que es representativo de una población mayor con características similares. Asumimos que existe una distribución de probabilidad verdadera $p^*(x)$ que describe cómo se generan los datos en esta población.

Nuestro objetivo es aproximar esta distribución verdadera mediante un modelo probabilístico parametrizado por θ , es decir:

$$p^*(x) \approx p_\theta(x)$$

Si el modelo se ha ajustado correctamente, entonces para una muestra común del conjunto, x_{normal} , se espera que $p_\theta(x_{\text{normal}})$ sea alto, mientras que para una muestra anómala, x_{outlier} , se espera que $p_\theta(x_{\text{outlier}})$ sea bajo. Esta función $p_\theta(x)$ es conocida como “verosimilitud” (likelihood), y el procedimiento de encontrar los parámetros θ que la maximizan se denomina *Maximum Likelihood Estimation* (MLE). Una introducción más conceptual puede encontrarse en [45].

A.2 MODELO LATENTE GENERATIVO

Para modelar $p_\theta(x)$, introducimos una variable latente $z \in \mathbb{R}^d$, que representa una codificación de menor dimensión que captura las características más relevantes de los datos. El modelo generativo define una distribución conjunta:

$$p_\theta(x, z) = p_\theta(z) \cdot p_\theta(x|z),$$

donde $p_\theta(z)$ es el prior sobre el espacio latente, usualmente una gaussiana estándar: $\mathcal{N}(\mathbf{0}, \mathbf{I})$ y $p_\theta(x|z)$ es el likelihood de los datos dado el vector de variables latentes z .

La probabilidad marginal (marginal likelihood) de los datos se obtiene integrando sobre todas las posibles configuraciones latentes:

$$p_{\theta}(x) = \int p_{\theta}(x, z) dz = \int p_{\theta}(z) \cdot p_{\theta}(x|z) dz$$

Nuestro interés radica en inferir la distribución posterior $p_{\theta}(z|x)$, es decir, la distribución del código latente dado un dato observado. Para ello, aplicamos la **regla de Bayes**:

$$p_{\theta}(z|x) = \frac{p_{\theta}(x|z) \cdot p_{\theta}(z)}{p_{\theta}(x)}$$

Esta expresión describe cómo actualizar nuestras creencias sobre la variable latente z al observar un dato x , combinando la verosimilitud $p(x|z)$ y el prior $p(z)$. Sin embargo, el término en el denominador $p(x)$ requiere calcular la integral marginal mostrada anteriormente, la cual es generalmente imposible de calcular de forma exacta.

Por esta razón, se recurre a técnicas de **inferencia variacional**, en las que se introduce una distribución aproximada $q_{\phi}(z|x)$ para estimar la posterior verdadera.

A.3 INFERENCIA VARIACIONAL AMORTIGUADA

La inferencia variacional consiste en aproximar la distribución posterior verdadera $p_{\theta}(z|x)$, típicamente intratable, mediante una familia de distribuciones variacionales $q_{\phi}(z|x)$ parametrizadas por ϕ . En el contexto del Autoencoder Variacional (VAE), esta aproximación se realiza de forma *amortiguada*, lo que implica que una red neuronal (el encoder) aprende a inferir directamente los parámetros de $q_{\phi}(z|x)$ para cada observación x , compartiendo los parámetros ϕ a lo largo del conjunto de datos.

El objetivo del VAE es maximizar la verosimilitud marginal $\log p_{\theta}(x)$; sin embargo, dado que esta es intratable en general, se optimiza una cota inferior conocida como *Evidence Lower Bound* (ELBO). La expresión fundamental que da origen al enfoque variacional es la siguiente descomposición:

$$\log p_{\theta}(x) = \text{ELBO}(x) + D_{\text{KL}}(q_{\phi}(z|x) \| p_{\theta}(z|x)) \quad (\text{A.1})$$

donde la ELBO está definida como:

$$\text{ELBO}(x) = \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{\text{KL}}(q_{\phi}(z|x) \| p_{\theta}(z)) \quad (\text{A.2})$$

Dado que la divergencia de Kullback–Leibler (KL) entre $q_{\phi}(z|x)$ y $p_{\theta}(z|x)$ es siempre no negativa, la ELBO constituye una cota inferior de $\log p_{\theta}(x)$. Así, maximizar la ELBO equivale a minimizar dicha divergencia, aproximando mejor la verdadera posterior y optimizando simultáneamente la reconstrucción de los datos. Para una representación visual de este concepto ver la [Figura 1.1](#).

La función objetivo ELBO consta de dos términos:

- **Término de reconstrucción:** representa la esperanza del logaritmo de la verosimilitud condicional $p_{\theta}(x|z)$, e indica qué tan bien el modelo puede reconstruir los datos observados x a partir de las variables latentes z .
- **Término de regularización (divergencia Kullback-Leibler (KL)):** mide la discrepancia entre la distribución aproximada $q_{\phi}(z|x)$ y el prior sobre el espacio latente $p_{\theta}(z)$, típicamente una gaussiana estándar $\mathcal{N}(0, \mathbf{I})$.

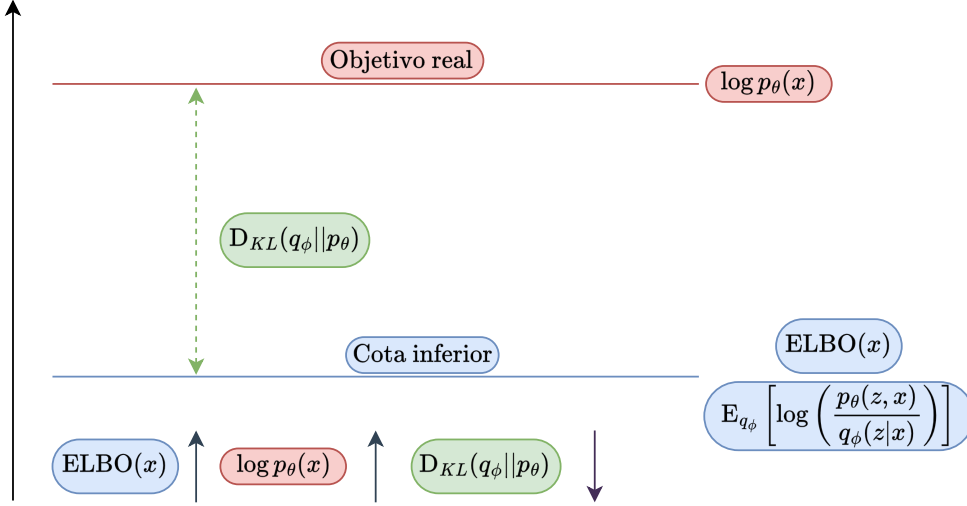


Figura 1.1: Maximizar la ELBO implica disminuir la divergencia KL entre la distribución aproximada $q_\phi(z|x)$ y la verdadera posterior $p_\theta(z|x)$, a la vez que se incrementa la probabilidad de los datos $p_\theta(x)$. Imagen adaptada de [46].

La **divergencia de Kullback-Leibler** $D_{KL}(q||p)$ es una medida asimétrica que cuantifica cuánta información se pierde al usar $q(z)$ para aproximar $p(z)$. Sus principales propiedades son:

- Es siempre no negativa: $D_{KL}(q||p) \geq 0$, y es igual a cero si y sólo si $q = p$ casi en todas partes.
- No es simétrica: $D_{KL}(q||p) \neq D_{KL}(p||q)$, lo que implica que el orden de las distribuciones importa.
- Penaliza fuertemente cuando q asigna probabilidad en regiones donde p es muy baja o nula.

La optimización conjunta de la ELBO permite que el modelo aprenda simultáneamente una representación latente significativa y una aproximación útil a la distribución de los datos, sin necesidad de evaluar directamente la posterior verdadera.

A.4 OPTIMIZACIÓN DE LA ELBO Y TRUCO DE LA REPARAMETRIZACIÓN

Para maximizar la ELBO, necesitamos calcular su gradiente respecto a los parámetros ϕ del encoder y θ del decoder. Sin embargo, el término de reconstrucción incluye una esperanza sobre la variable aleatoria $z \sim q_\phi(z|x)$, lo que impide propagar gradientes directamente a través de la muestra.

Para resolver este problema, se introduce el **truco de la reparametrización** [3], que permite reescribir la variable latente z como una función determinista de $\mu(x)$, $\sigma(x)$ y una variable auxiliar $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$z = \mu(x) + \sigma(x) \odot \varepsilon \quad (\text{A.3})$$

donde $\odot$ denota el producto elemento a elemento.

Esta reparametrización permite que el muestreo de z sea un paso determinista diferenciable con respecto a ϕ , lo cual posibilita optimizar la ELBO usando retropropagación estándar [47, 48, 49].

Entonces, la ELBO se estima de manera estocástica, es decir, mediante una aproximación basada en muestreo aleatorio sobre la variable latente z , ya que no se puede calcular exactamente de forma analítica, como:

$$\text{ELBO}(x) \approx \frac{1}{L} \sum_{l=1}^L \log p_{\theta}(x|z^{(l)}) - D_{\text{KL}}(q_{\phi}(z|x) \| p_{\theta}(z)) \quad (\text{A.4})$$

donde cada $z^{(l)}$ se genera usando la reparametrización, y L es el número de muestras Monte Carlo tomadas de la distribución aproximada $q_{\phi}(z|x)$. En teoría, tomar múltiples muestras por dato (valores altos de L) mejora la estimación de la esperanza, ya que reduce la varianza de la aproximación. Sin embargo, en la práctica se suele usar $L = 1$, porque el uso de batches grandes durante el entrenamiento ya produce un efecto de promediado similar: al calcular la pérdida como el promedio sobre muchos datos distintos, el ruido estocástico introducido por la muestra única de z para cada punto tiende a cancelarse, lo que resulta en una estimación global de la ELBO suficientemente estable y con baja varianza.

Gracias a este truco, la ELBO se puede optimizar directamente mediante algoritmos de gradiente estocástico como Adam.

A.5 AMORTIZACIÓN Y SUPOSICIONES SOBRE LA POSTERIOR

Para completar el cálculo de la ELBO, observamos que si bien el término de reconstrucción debe estimarse de manera estocástica mediante muestreo, la divergencia de Kullback–Leibler puede calcularse explícitamente gracias a la forma asumida para la distribución $q_{\phi}(z|x)$. Esta elección no es arbitraria, sino que está motivada por razones tanto computacionales como estadísticas.

En el contexto del VAE, el proceso de inferencia amortiguada consiste en parametrizar la distribución posterior aproximada $q_{\phi}(z|x)$ mediante una red neuronal (el encoder), cuyos parámetros ϕ son compartidos para todas las muestras del conjunto de datos.

Esta distribución se modela comúnmente como una gaussiana multivariante con matriz de covarianza diagonal:

$$q_{\phi}(z|x) = \mathcal{N}(z | \mu(x), \text{diag}(\sigma^2(x))) \quad (\text{A.5})$$

Este supuesto permite obtener resultados sin necesidad de recurrir a métodos numéricos o aproximaciones (fórmula cerrada) para el cálculo de la divergencia KL entre la distribución aproximada y la prior $p(z) = \mathcal{N}(0, I)$, lo que evita la necesidad de muestreo adicional y reduce la varianza de los gradientes. Así, la optimización de la ELBO se facilita sustancialmente.

Además, esta elección implica las siguientes características:

- Las dimensiones del vector latente z son **mutuamente independientes**,
- No se modelan correlaciones entre las variables latentes; la matriz de covarianza es estrictamente diagonal.

Este enfoque corresponde a la inferencia variacional del tipo *mean-field*, que, si bien es computacionalmente eficiente, impone una restricción sobre la flexibilidad de la familia de distribuciones utilizada para aproximar la posterior verdadera $p_{\theta}(z|x)$.

El objetivo durante la optimización es ajustar los parámetros ϕ de forma que la distancia entre la distribución aproximada $q_{\phi}(z|x)$ y la posterior verdadera $p_{\theta}(z|x)$ se minimice lo más posible. Aunque no se puede representar toda la complejidad de la verdadera posterior, esta aproximación permite una solución manejable y escalable al problema de inferencia bayesiana en modelos generativos profundos.

Finalmente, considerando que $q_\phi(z|x)$ es una gaussiana diagonal y que $p(z) = \mathcal{N}(0, I)$, la divergencia KL puede calcularse analíticamente como:

$$D_{\text{KL}}(q_\phi(z|x) \| p(z)) = \frac{1}{2} \sum_{j=1}^d (\mu_j^2(x) + \sigma_j^2(x) - \log \sigma_j^2(x) - 1) \quad (\text{A.6})$$

Por lo tanto, modificando la Ec. A.4, la función de pérdida que se minimiza durante el entrenamiento del VAE, $\mathcal{L}$, toma la forma:

$$-\mathcal{L}(x) = \text{ELBO}(x) \approx \frac{1}{L} \sum_{l=1}^L \log p_\theta(x|z^{(l)}) - \frac{1}{2} \sum_{j=1}^d (\mu_j^2(x) + \sigma_j^2(x) - \log \sigma_j^2(x) - 1) \quad (\text{A.7})$$

donde el primer término representa el **error de reconstrucción** (estimado por muestreo), y el segundo corresponde a la **regularización KL**, que penaliza desviaciones de la prior estándar.

A.6 CORRESPONDENCIA ENTRE LA FUNCIÓN DE PÉRDIDA Y LA ELBO

Durante el entrenamiento del VAE, se busca maximizar la cota inferior de la evidencia (ELBO), la cual se aproxima en la práctica mediante una función de pérdida que puede ser implementada computacionalmente. En este caso, la pérdida total se compone de dos términos: el error de reconstrucción y la divergencia KL.

El **término de reconstrucción** estima la verosimilitud $\log p_\theta(x|z)$, y se calcula utilizando el valor reconstruido por el decodificador.

CASO ENTRADA REAL. Si las variables de entrada son **reales**, es común modelar esta distribución como una gaussiana con media $\hat{x}$ y **varianza fija e isotrópica** (es decir, igual para todas las dimensiones y constante en el tiempo). Esta elección no sólo simplifica la implementación y el cálculo del término de log-verosimilitud, sino que también actúa como una forma de regularización: mantener la varianza constante evita que el modelo aprenda a asignar varianzas extremadamente pequeñas para forzar una reconstrucción perfecta, lo que conduciría a sobreajuste.

En este contexto, la log-verosimilitud se reduce (ignorando constantes aditivas) a una medida de error cuadrático medio (MSE):

$$\text{Reconstrucción} \approx \|x - \hat{x}\|^2$$

Así, la función de pérdida que se minimiza durante el entrenamiento es simplemente el negativo de la ELBO:

$$\mathcal{L}_{\text{total}}(x) = \underbrace{\|x - \hat{x}\|^2}_{\text{Reconstrucción}} + \underbrace{\frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)}_{\text{Divergencia KL}} \quad (\text{A.8})$$

En el código, estos términos se calculan con funciones estándar de TensorFlow:

- `MeanSquaredError` para la reconstrucción.

- Una expresión cerrada para la KL que depende de z_mean y z_log_var .

CASO ENTRADA BINARIA. Si las entradas x son **variables binarias** (por ejemplo, imágenes en blanco y negro o vectores one-hot), entonces la distribución de salida se modela como una **Bernoulli**, y el término de reconstrucción cambia a una pérdida binaria cruzada (binary cross-entropy):

$$\log p_{\theta}(x|z) \approx \sum_{i=1}^n [x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i)] \quad (\text{A.9})$$

En este caso, la función de pérdida total se vuelve:

$$\mathcal{L}_{\text{total}}(x) = \underbrace{\text{BCE}(x, \hat{x})}_{\text{Reconstrucción (binaria)}} + \underbrace{\frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)}_{\text{Divergencia KL}} \quad (\text{A.10})$$

Este ajuste asegura que la reconstrucción se evalúe con la función de pérdida adecuada al tipo de datos, manteniendo el marco general del VAE.

INICIALIZACIÓN DE PESOS

La correcta inicialización de los pesos en redes neuronales profundas es fundamental para garantizar una propagación efectiva de los gradientes durante el entrenamiento, evitando problemas como el desvanecimiento o explosión de gradientes. En el diseño del modelo propuesto, se seleccionaron esquemas de inicialización específicos para cada tipo de capa, en función de la activación utilizada y el papel que cumple dentro del VAE.

B.1 RESUMEN DE ESQUEMAS UTILIZADOS

Se emplearon tres estrategias principales de inicialización:

- **He Normal**, para capas con activación ReLU.
- **Glorot Uniform (Xavier)**, para la capa que estima la media de la distribución latente z_{mean} .
- **LeCun Normal**, para la capa que estima la log-variancia $z_{\text{log var}}$.

A continuación, se detalla el fundamento teórico de cada esquema y su pertinencia en el contexto del modelo.

B.2 HE NORMAL (PARA RELU)

La activación *ReLU* (Rectified Linear Unit) presenta la particularidad de que para entradas negativas, su salida es cero, y para entradas positivas, la salida es lineal. Este comportamiento puede inducir una varianza de salida muy baja si no se inician adecuadamente los pesos, afectando negativamente el aprendizaje [50].

Para mitigar este problema, se utiliza la inicialización **He Normal**, diseñada específicamente para capas con ReLU. Esta técnica ajusta la varianza de los pesos de acuerdo con el número de entradas n_{in} de la capa, asegurando una propagación estable de los gradientes.

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

Este escalamiento garantiza que la varianza de las salidas se mantenga igual a la de las entradas, facilitando una activación eficiente en redes profundas. Aunque el modelo utilizado no es extremadamente profundo, se considera ventajoso emplear ReLU y su inicialización adecuada para acelerar la convergencia.

B.3 GLOT UNIFORM (PARA z_{mean})

En el caso de la capa encargada de estimar la media de la distribución latente, z_{mean} , se utiliza una activación lineal. Para este tipo de activación, es importante que los pesos iniciales no induzcan activaciones ni demasiado grandes ni demasiado pequeñas, lo que podría sesgar el entrenamiento.

Se opta entonces por la inicialización **Glorot Uniform** [51], también conocida como **Xavier**, que busca equilibrar la varianza entre entradas y salidas.

$$W \sim \mathcal{U} \left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}} \right)$$

Este esquema favorece una convergencia más rápida y estable en redes con activaciones lineales, evitando que las activaciones se desborden o se atenúen excesivamente.

B.4 LECUN NORMAL (PARA $z_{\text{log_var}}$)

La estimación de la log-variancia $z_{\text{log_var}}$ es particularmente sensible, ya que cualquier desbalance en su valor puede afectar directamente la validez de la distribución gaussiana generada, debido a que la varianza debe ser siempre positiva y moderada.

Para este caso se emplea la inicialización **LeCun Normal** [52], diseñada para activaciones suaves, como la función lineal empleada aquí. Su fórmula es:

$$W \sim \mathcal{N} \left(0, \frac{1}{n_{\text{in}}} \right)$$

Este tipo de inicialización minimiza el riesgo de obtener valores extremos al aplicar la exponenciación a $z_{\text{log_var}}$, lo cual es crucial para garantizar una varianza finita, positiva y estable durante el muestreo del espacio latente. De este modo, se favorece la convergencia del modelo y la estabilidad numérica del entrenamiento.

EXPLORACIÓN DETALLADA DEL CONJUNTO DE DATOS

El conjunto de datos utilizado en este trabajo contiene un total de 284,807 transacciones financieras, de las cuales 492 (0.172 %) han sido etiquetadas como fraudulentas. Cada transacción está representada por 30 variables numéricas: *Time*, que indica los segundos transcurridos desde la primera transacción registrada; *Amount*, que corresponde al monto monetario de la transacción; y *V1* a *V28*, que son variables resultantes de un análisis de componentes principales (PCA) aplicado sobre las variables originales, cuyo contenido se mantiene confidencial. Es importante destacar que no existen valores nulos en el conjunto, lo cual simplifica las tareas de preprocesamiento y asegura la integridad estadística de los análisis exploratorios.

C.1 ANÁLISIS EXPLORATORIO DE CARACTERÍSTICAS

Se llevó a cabo un análisis estadístico y visual de las variables del conjunto, mostrándose aquí algunas de ellas. Las características presentan medias de cero y desviaciones estándar de uno tras la estandarización mediante Z-score, lo que confirma la correcta transformación de los datos para su uso en modelos sensibles a escalas, como los autoencoders variacionales.

Desde el punto de vista estadístico, algunas observaciones relevantes incluyen:

- La variable *Time* presenta una media de 94,813 segundos y un amplio rango (0 a 172,792), lo que sugiere un registro continuo de transacciones a lo largo de dos días aproximadamente.
- Las variables transformadas por PCA (de *V1* a *V28*) están centradas en cero y en su mayoría dentro de un rango de ± 5 , aunque algunas exhiben valores extremos notables, como *V7* (máximo de 120.59) o *V20* (mínimo de -54.49).
- La variable *Class* presenta un fuerte desbalance de clases, con sólo el 0.17 % de las observaciones marcadas como fraude, aspecto crítico para los métodos de detección aplicados.

C.1.1 *Distribuciones diferenciadas por clase*

Para identificar patrones que distingan comportamientos normales de los fraudulentos, se analizaron distribuciones de variables clave segmentadas por clase. A continuación, se presentan gráficos KDE que revelan diferencias notables:

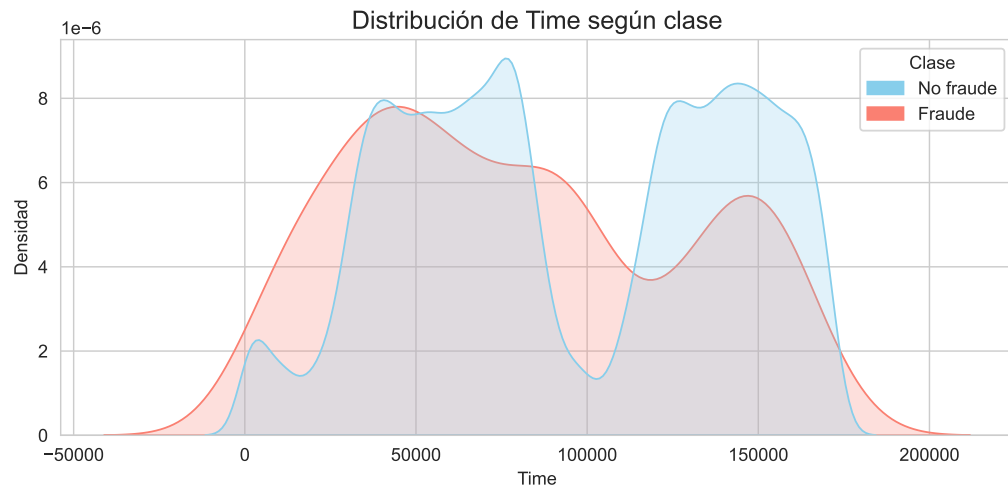


Figura 3.1: Distribución de Time diferenciada por clase. Las transacciones no fraudulentas tienden a concentrarse en ciertas ventanas temporales.

Las variables transformadas por PCA también evidencian estructuras diferenciadas entre clases, como se muestra a continuación:

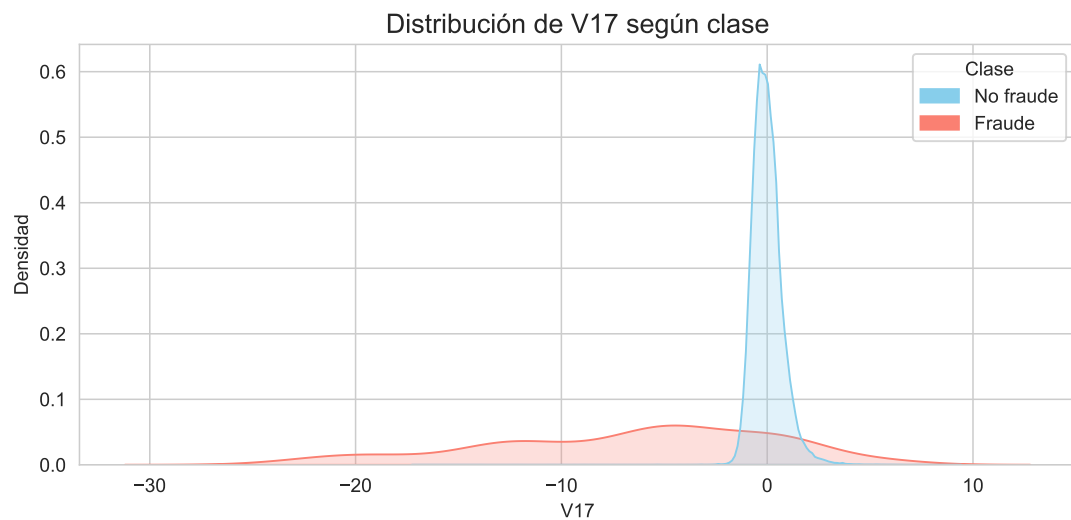


Figura 3.2: Distribución de la variable V17 por clase. Las transacciones fraudulentas presentan una concentración marcada en torno al valor cero, lo que sugiere una estructura anómala más rígida. En contraste, las transacciones normales exhiben una distribución más dispersa y simétrica, con valores que se extienden hacia ambos extremos. Esta diferencia en la forma y dispersión puede ser indicativa de la utilidad de V17 para distinguir entre clases.

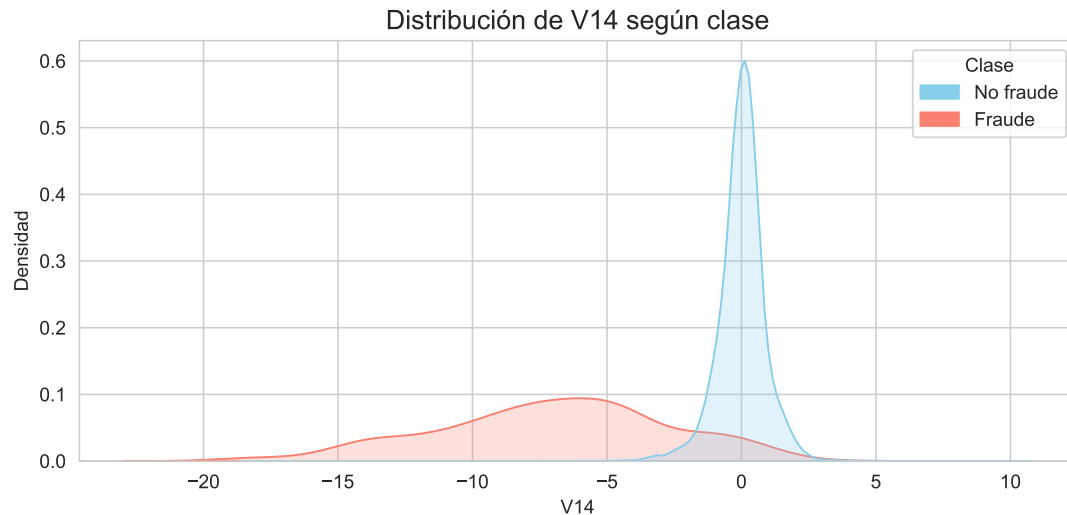


Figura 3.3: Distribución de V14 por clase. Los valores fraudulentos se concentran en zonas negativas.

Además de los gráficos de densidad, se emplearon representaciones adicionales para profundizar en la comprensión de la distribución de las variables según la clase. En particular, se utilizaron gráficos de tipo *violin plot*, que combinan información de distribución y densidad, y *box plots*, que permiten visualizar la mediana, los cuartiles y posibles valores atípicos.

Por ejemplo, en la [Figura 3.4](#) se aprecia la marcada asimetría en la distribución de la variable V17 entre transacciones normales y fraudulentas, lo cual sugiere su potencial relevancia como variable discriminativa. Asimismo, en la [Figura 3.5](#) se observa cómo la variable V14 presenta diferencias sustanciales en su dispersión y mediana según la clase, lo que refuerza su utilidad para tareas de clasificación.



Figura 3.4: *Violin plot* de la variable V17 por clase. La asimetría en la distribución refuerza su relevancia para separar clases.

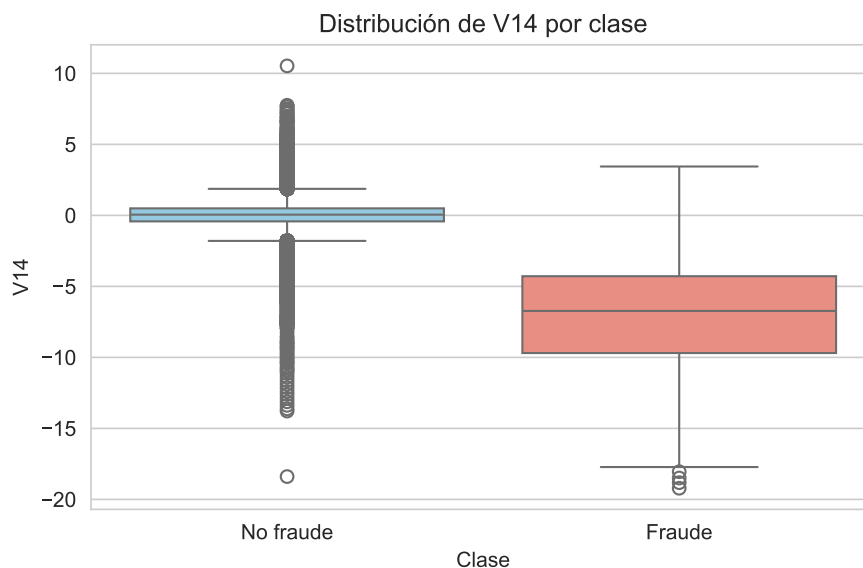


Figura 3.5: Box plot de la variable V14 por clase. Las diferencias en la mediana y dispersión sugieren capacidad discriminativa.

c.1.2 Relación multivariada con la variable objetivo

Finalmente, se analizaron las correlaciones entre todas las variables y la clase, con el fin de identificar aquellas que podrían aportar mayor valor predictivo. En la Figura 3.6 se presenta un mapa de calor que muestra las cinco variables con mayor correlación absoluta con Class. Este análisis resulta útil para establecer umbrales variables en la detección de anomalías según la categoría, permitiendo focalizarse en aquellas variables que evidencian un comportamiento más relevante para la clasificación.

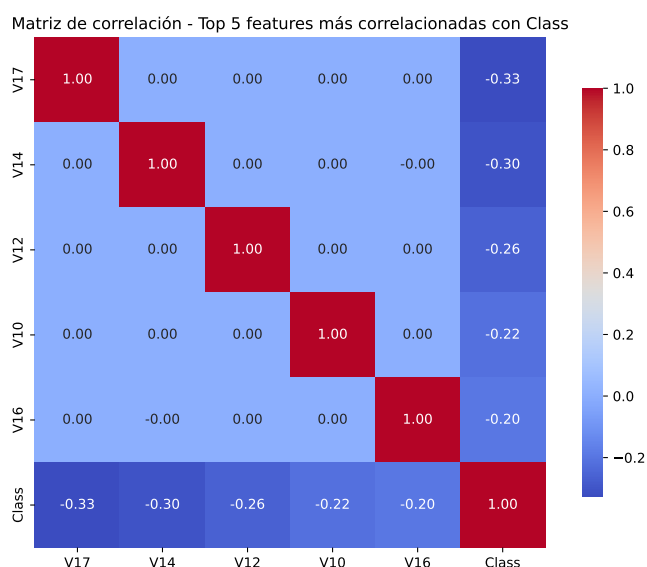


Figura 3.6: Matriz de correlación entre las cinco variables más correlacionadas con la clase. Se observa una relación significativa, especialmente en variables como V17, V14 y V12.

DETECCIÓN DE ANOMALÍAS Y RECONSTRUCCIÓN

Una de las características más relevantes de los modelos *Variational Autoencoder* (VAE) es la capacidad de regular el equilibrio entre la fidelidad de reconstrucción y la regularización del espacio latente a través del hiperparámetro β .

A medida que disminuimos el valor de β , se relaja la presión sobre la distribución latente para ajustarse a una distribución prior (usualmente una normal estándar), lo que permite una representación más dispersa y menos regularizada en el espacio latente. Este comportamiento acerca el modelo a un Autoencoder Clásico, privilegiando la precisión de reconstrucción. Como puede observarse en las figuras [Figura 4.1](#) y [Figura 4.2](#), al utilizar un $\beta = 0.09$ se obtienen reconstrucciones notablemente más precisas, tanto para las observaciones normales como anómalas.

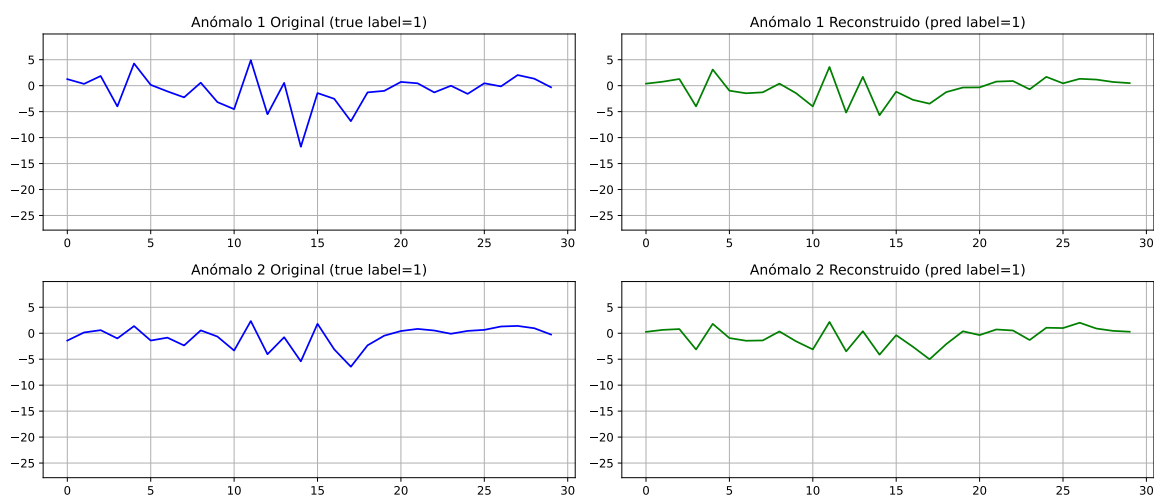


Figura 4.1: Reconstrucción (verde) de datos anómalos originales (azul) con $\beta = 0.09$. Se logra una mayor fidelidad de reconstrucción.

En contraste, cuando se emplea un valor más alto de β , como $\beta = 0.9$, la regularización del espacio latente domina el proceso de entrenamiento, sacrificando fidelidad en la reconstrucción. Esto se traduce en reconstrucciones menos detalladas o directamente erróneas, como puede verse en las figuras [Figura 4.3](#) y [Figura 4.4](#).

Lo interesante surge al analizar el impacto de estas diferencias en la tarea de detección de anomalías. Contrario a lo que podría esperarse, el modelo que reconstruye mejor (con $\beta = 0.09$) tiende a detectar peor las anomalías. Esto se observa claramente en las matrices de confusión correspondientes ([Figura 4.5](#) y [Figura 4.6](#)). El modelo

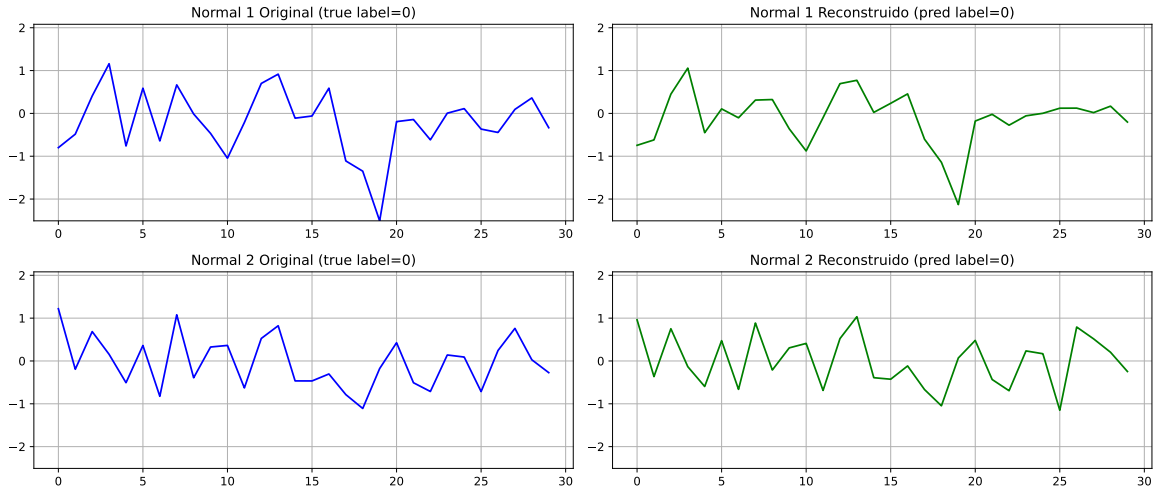


Figura 4.2: Reconstrucción (verde) de datos normales originales (azul) con $\beta = 0.09$. Se observan reconstrucciones de alta precisión.

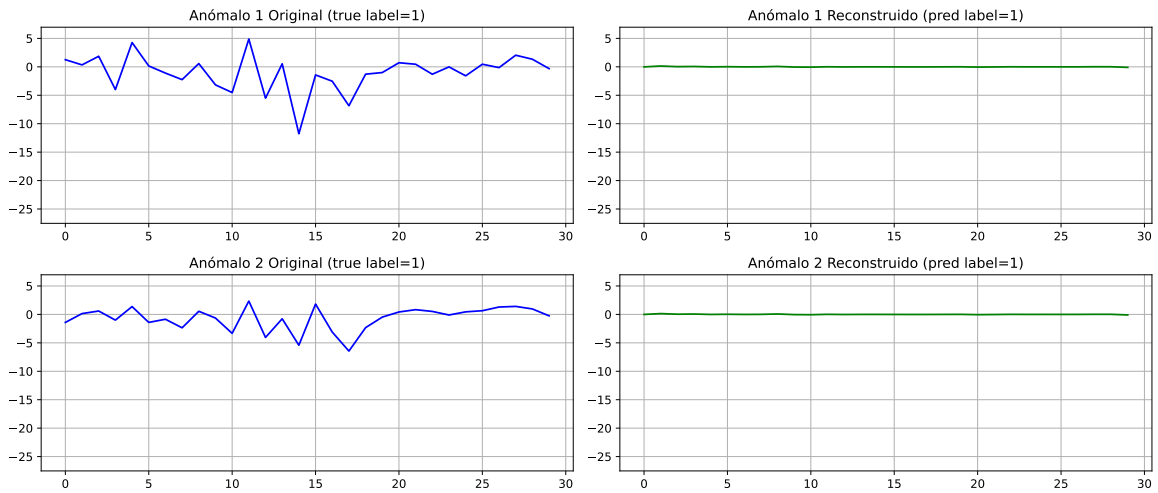


Figura 4.3: Reconstrucción (verde) de datos anómalos originales (azul) con $\beta = 0.9$. Se observa una baja fidelidad en la reconstrucción.

con $\beta = 0.9$, pese a su peor desempeño en reconstrucción, logra una mejor discriminación entre datos normales y anómalos, dado que la diferencia en los errores de reconstrucción entre ambos grupos es más marcada.

Este comportamiento también se refleja en los umbrales de decisión óptimos para cada modelo. El modelo con $\beta = 0.09$ requiere un umbral de 0.87, mientras que el modelo con $\beta = 0.9$ utiliza un umbral significativamente mayor, de 1.45. Este ajuste responde a la necesidad de adaptarse a los errores de reconstrucción más altos que produce el modelo con mayor regularización, lo que paradójicamente genera una mayor fidelidad en la detección de anomalías. Además, como los datos fueron previamente estandarizados, la raíz cuadrada del umbral puede interpretarse como una medida de desviación estándar respecto del valor original reconstruido. Así, $\sqrt{0.87} \approx 0.93$ y $\sqrt{1.45} \approx 1.20$, lo que implica que se considera anómalo todo punto cuya reconstrucción difiere del original en más de aproximadamente 0.93 y 1.20 desviaciones estándar, respectivamente.

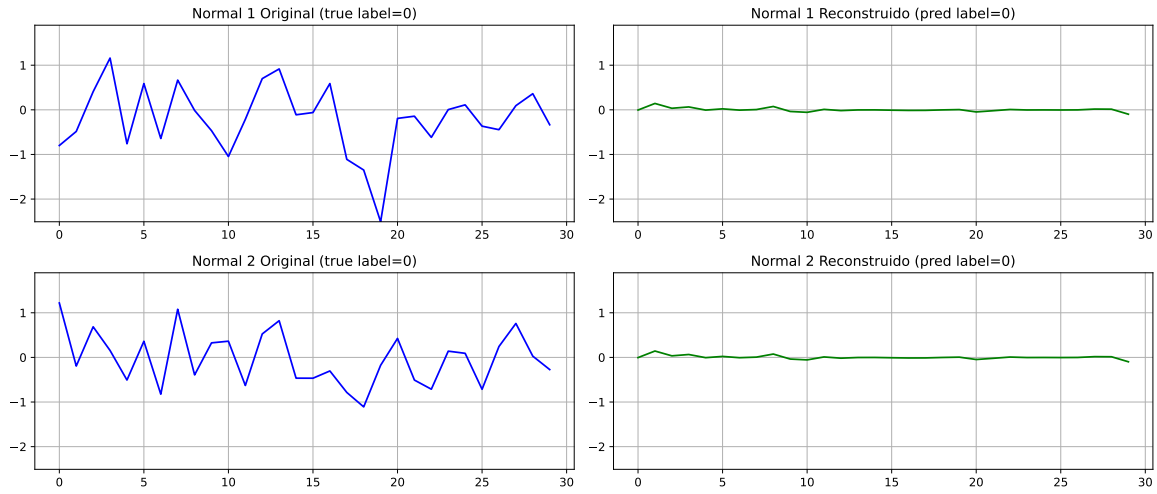


Figura 4.4: Reconstrucciones (verde) de datos normales originales (azul) con $\beta = 0.9$. Las reconstrucciones presentan distorsiones notables.

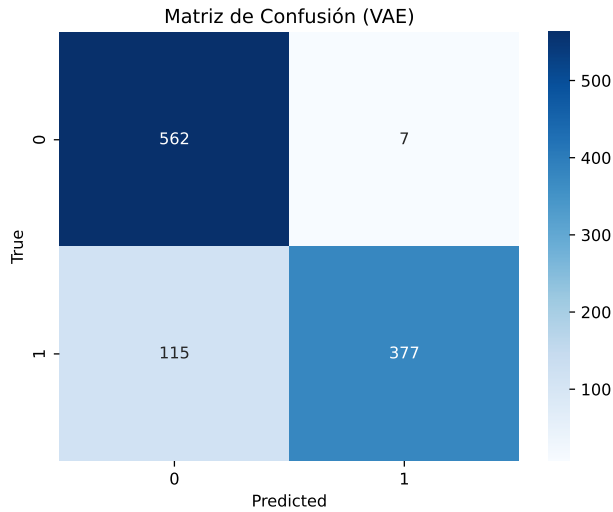


Figura 4.5: Matriz de confusión para $\beta = 0.09$, con un umbral de 0.87. A pesar de las buenas reconstrucciones, la separación entre normales y anómalos es menos efectiva.

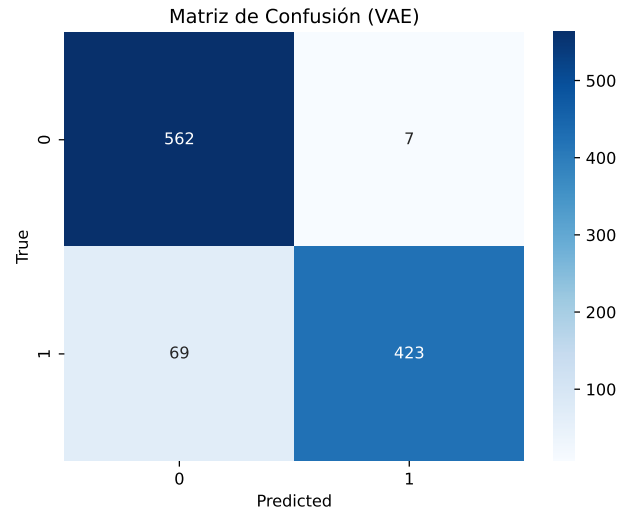


Figura 4.6: Matriz de confusión para $\beta = 0.9$, utilizando un umbral de 1.45. A pesar de las reconstrucciones deficientes, se logra una buena separación entre normales y anómalos.

BIBLIOGRAFÍA

- [1] Zhaomin Chen, Chai Kiat Yeo, Bu Sung Lee, and Chiew Tong Lau. Autoencoder-based network anomaly detection. In *2018 Wireless Telecommunications Symposium (WTS)*, pages 1–5, 2018.
- [2] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-vae: Learning basic visual concepts with a constrained variational framework. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.
- [3] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [4] Jinwon An and Sungzoon Cho. Variational autoencoder based anomaly detection using reconstruction probability. *Special lecture on IE*, 2(1):1–18, 2015.
- [5] M Shashanka et al. Anomaly detection in user activities using mahalanobis distance and svd. *Proceedings of the First ACM Workshop on Artificial Intelligence and Security*, 2008.
- [6] John Voris et al. Application of gmm in host-based anomaly detection. *Digital Investigation*, 2013.
- [7] M Pusara and CE Brodley. Using graph-based user profiles for identity theft detection. In *Proceedings of the First International Workshop on Recent Advances in Intrusion Detection*, 2004.
- [8] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [9] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996.
- [10] Arthur E Hoerl and Robert W Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- [11] Yingce Xia, Lili Li, Tao Liu, Fei Wang, Dacheng Tao, and Tie-Yan Liu. Robust regression via deep neural networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04):6473–6480, 2020.
- [12] Bernhard Schölkopf, John C Platt, John Shawe-Taylor, Alexander J Smola, and Robert C Williamson. Estimating the support of a high-dimensional distribution. *Neural computation*, 13(7):1443–1471, 2001.
- [13] Shehroz S Khan and Michael G Madden. One-class classification: taxonomy of study and review of techniques. *The Knowledge Engineering Review*, 29(3):345–374, 2014.
- [14] Markus M Breunig, Hans-Peter Kriegel, Raymond T Ng, and Jörg Sander. Lof: Identifying density-based local outliers. *ACM sigmod record*, 2000.
- [15] Guilherme O Campos, Arthur Zimek, Jörg Sander, Ricardo JGB Campello, Barbora Micenková, Erich Schubert, Ira Assent, and Michael E Houle. On the evaluation of unsupervised outlier detection: measures, datasets, and an empirical study. *Data Mining and Knowledge Discovery*, 30:708–749, 2016.

- [16] Sridhar Ramaswamy, Rajeev Rastogi, and Kyuseok Shim. Efficient algorithms for mining outliers from large data sets. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, pages 427–438, 2000.
- [17] Stephen D Bay and Mark Schwabacher. The nearest neighbor anomaly detection algorithm: a comparative study. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2003.
- [18] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- [19] Anirban Ghosh, Pei Li, Vishnu Kulkarni, Suyash Gupta, and Prasenjit Mitra. Understanding decision forest representations for anomaly detection. *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 1355–1364, 2020.
- [20] X. Liu, Y. Zhang, and Y. Huang. Anomaly detection in high-dimensional data using decision tree ensembles. *Expert Systems with Applications*, 193:116501, 2022.
- [21] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. *2008 Eighth IEEE International Conference on Data Mining*, 2008.
- [22] Sahand Hariri, Maximilian Kind, and Robert J Brunner. Extended isolation forest. *IEEE Transactions on Knowledge and Data Engineering*, 2021.
- [23] Raghavendra Chalapathy and Sanjay Chawla. Deep learning for anomaly detection: A survey. *ACM Computing Surveys (CSUR)*, 51(3):1–36, 2019.
- [24] M Sakurada and T Yairi. Anomaly detection using autoencoders with satellite data. *Proceedings of the 2014 ACM Symposium on Applied Computing*, 2014.
- [25] C Zhou and R Paffenroth. Anomaly detection with robust autoencoders. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017.
- [26] Li Wang, Yifan Zhang, and Yu Chen. A comprehensive survey on deep learning-based anomaly detection in industrial control systems. *IEEE Access*, 10:2173–2199, 2022.
- [27] S Hawkins et al. Outlier detection using autoencoders. *Proceedings of the Fourth International Conference on Data Warehousing and Knowledge Discovery*, 2002.
- [28] Jose Fuentes, Ines Ortega-Fernandez, Nora M. Villanueva, and Marta Sestelo. Cybersecurity threat detection based on a ueba framework using deep autoencoders, 2025.
- [29] B Ribeiro et al. Video anomaly detection with convolutional autoencoders. *Pattern Recognition Letters*, 2018.
- [30] Pankaj Malhotra, Lovekesh Vig, Gautam Shroff, and Puneet Agarwal. Lstm-based encoder-decoder for multi-sensor anomaly detection. *arXiv preprint arXiv:1607.00148*, 2016.
- [31] Camilo Piñón-Blanco, Fabián Otero-Vázquez, Ines Ortega-Fernandez, and Marta Sestelo. Detecting anomalies in industrial control systems with LSTM neural networks and UEBA. In *2023 JNIC Cybersecurity Conference (JNIC)*. IEEE, jun 2023.

- [32] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103, 2008.
- [33] Efthimios Pantelidis, Gueltoum Bendiab, Stavros Shiaeles, and Nicholas Kolokotronis. Insider detection using deep autoencoder and variational autoencoder neural networks. *CoRR*, abs/2109.02568, 2021.
- [34] A González-Muñiz et al. Anomaly classification using vae on engineering sequences. *Engineering Applications of Artificial Intelligence*, 2020.
- [35] Z Wang et al. Vae with transformers for satellite telemetry. *IEEE Access*, 2021.
- [36] D Slipenchuk and A Epishkina. Ueba using biometrics for banking operations. *Information Security Journal*, 2019.
- [37] C Xiao et al. Vae for anomaly scoring. *Advances in Neural Information Processing Systems*, 2018.
- [38] D Morales-Forero and J Bassetto. Explainable anomaly detection in industrial data using ae and lstm. *IEEE Transactions on Industrial Informatics*, 2020.
- [39] Richard H. R. Hahnloser, R. Sarpeshkar, M. A. Mahowald, R. J. Douglas, and H. S. Seung. Digital selection and analogue amplification coexist in a cortex-inspired silicon circuit. *Nature*, 405(6789):947–951, 2000.
- [40] Kevin Jarrett, Koray Kavukcuoglu, Marc’Aurelio Ranzato, and Yann LeCun. What is the best multi-stage architecture for object recognition? In *Proceedings of the 12th IEEE International Conference on Computer Vision*, pages 2146–2153, 2009.
- [41] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, pages 807–814, 2010.
- [42] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 448–456, Lille, France, 2015. PMLR.
- [43] Sepp Hochreiter. The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(02):107–116, 1998.
- [44] Andrea Dal Pozzolo, Olivier Caelen, Yann-Aël Le Borgne, Sébastien Waterschoot, and Gianluca Bontempi. Credit card fraud detection dataset. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>, 2016. Accessed: 2025-06-17.
- [45] Stanford University. Cs229 lecture notes - autoencoders and variational autoencoders. Technical report, Stanford University, 2019. Disponible en <https://cs229.stanford.edu/summer2019/cs229-notes8.pdf>. Accedido: 15 de abril de 2025.
- [46] Luis Serrano. What is a variational autoencoder? (vae), 2017. Video disponible en YouTube: <https://www.youtube.com/watch?v=HBYQvKlaE0A>. Accedido: 15 de abril de 2025.
- [47] Timothy P Lillicrap, Adam Santoro, Luke Marris, Colin J Akerman, and Geoffrey Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.

- [48] ROBERT HECHT-NIELSEN. Iii.3 - theory of the backpropagation neural network**based on “nonindent” by robert hecht-nielsen, which appeared in proceedings of the international joint conference on neural networks 1, 593–611, june 1989. © 1989 ieee. In Harry Wechsler, editor, *Neural Networks for Perception*, pages 65–93. Academic Press, 1992.
- [49] Barry J. Wythoff. Backpropagation neural networks: A tutorial. *Chemometrics and Intelligent Laboratory Systems*, 18(2):115–155, 1993.
- [50] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, 2015.
- [51] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- [52] Yann LeCun, Leon Bottou, Genevieve B Orr, and Klaus-Robert Müller. Efficient backprop. In *Neural networks: Tricks of the trade*, pages 9–48. Springer, 2012.